

Sesión 3

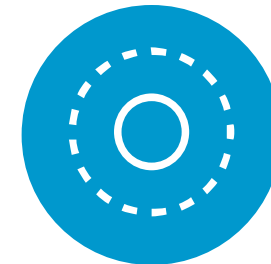
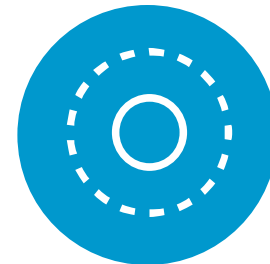
06 Mayo 2025

- 1. Material de Apoyo**
- 2. Apache Kafka**
- 3. BBDD NoSQL**
- 4. Redshift**

1/ Material de Apoyo

UF1 Aplicación de técnicas de integración, procesamiento y análisis de información.

L02. Técnicas y procesos de extracción de la información de los datos .

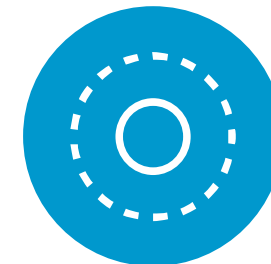
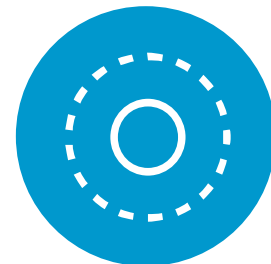


UF2 Técnicas de representación y minería de datos

L03. Métodos y algoritmos de minería de datos

UF 3 Sistemas de almacenamiento

Sistemas de gestión almacenamiento HDFS.
Apache Kafka, Motores y bbdd NoSQL



UF 4 Conceptos y visualizacion

L04 Herramientas de visualización de datos

- Página de Inicio
- Temas
- Programación semanal
- Archivos
- Clases en directo
- Anuncios
- Foros de discusión
- Tareas
- Calificaciones
- Calificaciones finales
- Personas
- Evaluaciones
- Módulos
- Páginas
- Programa del curso
- Competencias
- Colaboraciones

Programa Profesional en Inteligencia Artificial y Data Science - Octubre 2024 (PER 11324) > ... > Material de Apoyo

Buscar archivos

0 items seleccionados

+ Carpeta

Cargar

Programa Profesional en Inteligencia Artificial y Data Science

Actividades

Multimedia cargada

Presentaciones clases

Módulo 1: Capacidades de Inteligencia Artificial

Módulo 2: Lenguajes y Desarrollo

Módulo 3: Aprendizaje Automático

Módulo 4: Analítica Escalable

Material de Apoyo

Actividades de Evaluación

Actividad 7

Actividad 8

Docker Contenedores

Nombre	Fecha de creación	Fecha de modificación	Modificado por	Tamaño
Actividades de Evaluación	Martes			
Docker Contenedores	23 de abr de 2025			
Labs de AWS	27 de abr de 2025			
Practica HDFS	23 de abr de 2025			
Practicas Kafka	23 de abr de 2025			

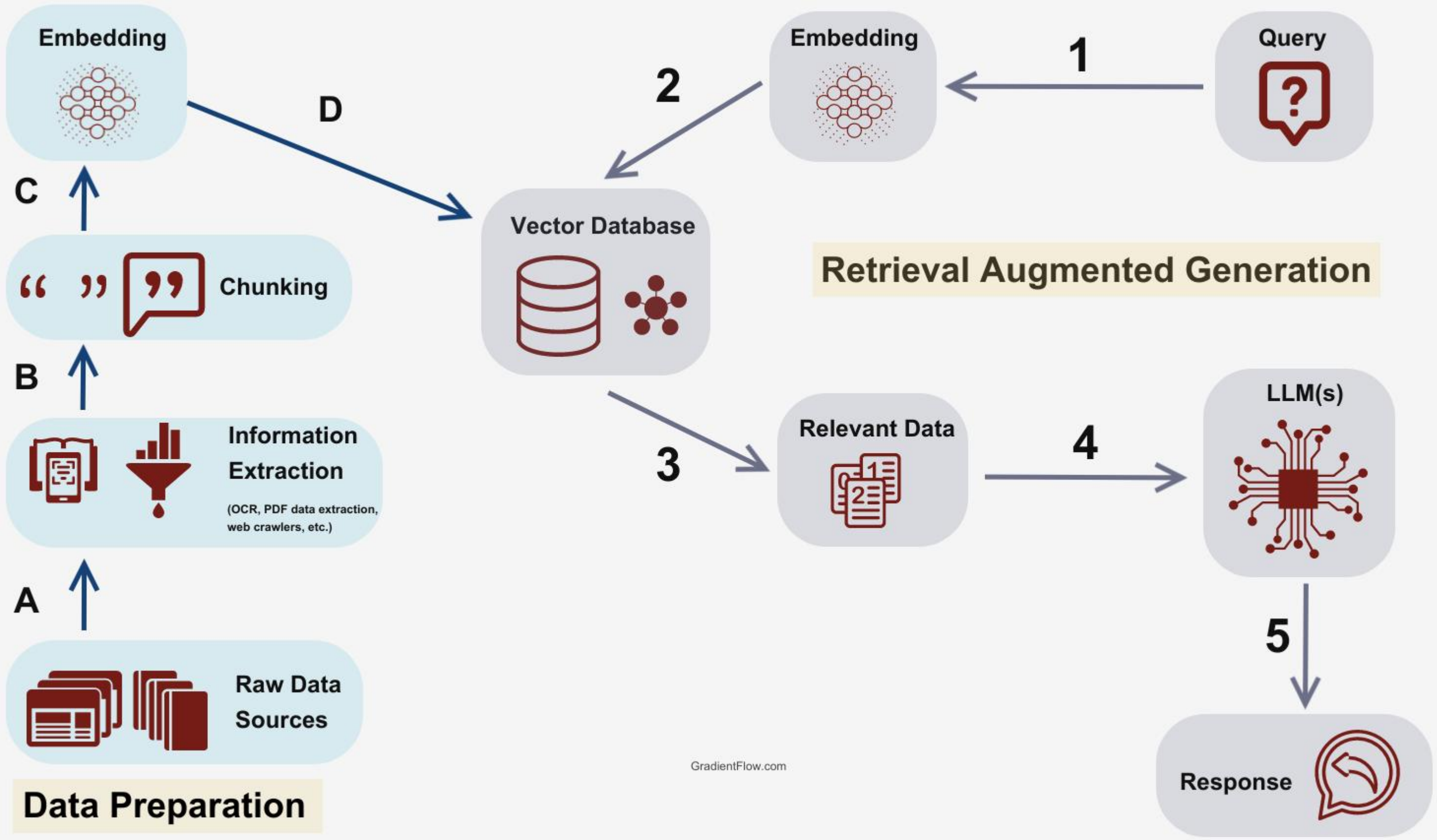
96% de 524 MB utilizados

Todos mis archivos

2/ Apache Kafka

¿Qué desafíos creen que existen al manejar grandes volúmenes de datos en tiempo real para la IA?





Apache Kafka es una **plataforma de mensajería distribuida** diseñada para manejar el procesamiento de flujos de datos en tiempo real. Kafka permite publicar, suscribirse, almacenar y procesar transmisiones de datos de manera eficiente y escalable. A diferencia de los sistemas de mensajería tradicionales, que son generalmente más simples y menos orientados a manejar grandes volúmenes de datos, Kafka se destaca por su **alta durabilidad, escalabilidad horizontal y bajo acoplamiento** entre los productores y consumidores de datos.

En términos simples, Kafka actúa como un "intermediario" o "buffer" que facilita la transmisión de datos (mensajes) entre sistemas que producen datos (producers) y sistemas que consumen o procesan esos datos (consumers).

Apache Kafka fue creado en **2011** por ingenieros de LinkedIn (Jay Kreps, Neha Narkhede y Jun Rao) para resolver un problema particular: la creciente necesidad de procesar enormes cantidades de datos generados en tiempo real dentro de la red social profesional. LinkedIn se enfrentaba a desafíos en la administración y procesamiento de flujos de eventos y datos de diversas fuentes, como actividad de usuarios, métricas operativas y registros de aplicaciones.

Antes de Kafka, LinkedIn, al igual que muchas otras empresas, usaba una combinación de bases de datos y herramientas tradicionales para manejar sus datos, pero estas soluciones no eran suficientemente rápidas ni eficientes para manejar el volumen creciente de información. Se necesitaba una solución que pudiera:

- **Escalar** de forma horizontal en cientos de servidores.
- Ofrecer **alta tolerancia a fallos**.
- Manejar grandes volúmenes de datos en tiempo real sin comprometer el rendimiento.
- Proporcionar una **baja latencia** en la entrega de mensajes.

Para resolver estos problemas, Apache Kafka fue diseñado con los principios de:

1.Almacenamiento distribuido: Los datos se distribuyen en múltiples servidores o "brokers", y se pueden replicar para aumentar la durabilidad.

2.Procesamiento en tiempo real: Kafka está optimizado para manejar y procesar datos a medida que se generan, en lugar de ser un sistema de procesamiento por lotes.

3.Alta escalabilidad: Kafka puede manejar millones de mensajes por segundo con una infraestructura adecuada, lo que lo convierte en una herramienta adecuada para sistemas de gran escala.

4.Persistencia: A diferencia de muchos sistemas de mensajería, Kafka almacena mensajes durante períodos de tiempo configurables, lo que permite a los consumidores leer mensajes antiguos según sea necesario.



Conceptos clave:

- **Mensajes (Records):** La unidad básica de datos.
- **Topics:** Canales o categorías donde se publican los mensajes.
- **Particiones:** División horizontal de un topic para paralelismo y escalabilidad.
- **Brokers:** Servidores que almacenan las particiones de los topics.
- **Producers:** Aplicaciones que publican mensajes en los topics.
- **Consumers:** Aplicaciones que se suscriben a los topics para consumir mensajes.
- **Zookeeper:** Servicio de coordinación para los brokers de Kafka.
- **Grupos de Consumidores:** Agrupaciones de consumidores que trabajan juntos para consumir mensajes de un topic, garantizando que cada mensaje sea procesado por solo un consumidor dentro del grupo. Un **grupo de consumidores (consumer group)** en Apache Kafka es un mecanismo fundamental para habilitar el **procesamiento paralelo y escalable** de los mensajes dentro de un topic. Permite que múltiples aplicaciones o instancias de la misma aplicación trabajen juntas para consumir todos los mensajes de un topic sin procesar el mismo mensaje duplicado.

Componentes Principales de Apache Kafka

Los principales actores en el sistema Kafka son:

a. Producers (Productores)

Los **producers** son las aplicaciones o servicios que **generan y envían mensajes** a Kafka. Los productores publican los mensajes en **topics** específicos. Algunas características clave de los producers:

Elección de particiones: Un productor puede decidir en qué partición de un topic específico almacenar un mensaje. Puede hacerlo aleatoriamente o utilizando una clave para garantizar que los mensajes relacionados vayan a la misma partición.

Confirmación de envío (acks): Los productores pueden configurar cómo Kafka debe confirmar la recepción de los mensajes. Por ejemplo, pueden esperar confirmación de todos los brokers o solo de uno para equilibrar rendimiento y durabilidad.

b. Consumers (Consumidores)

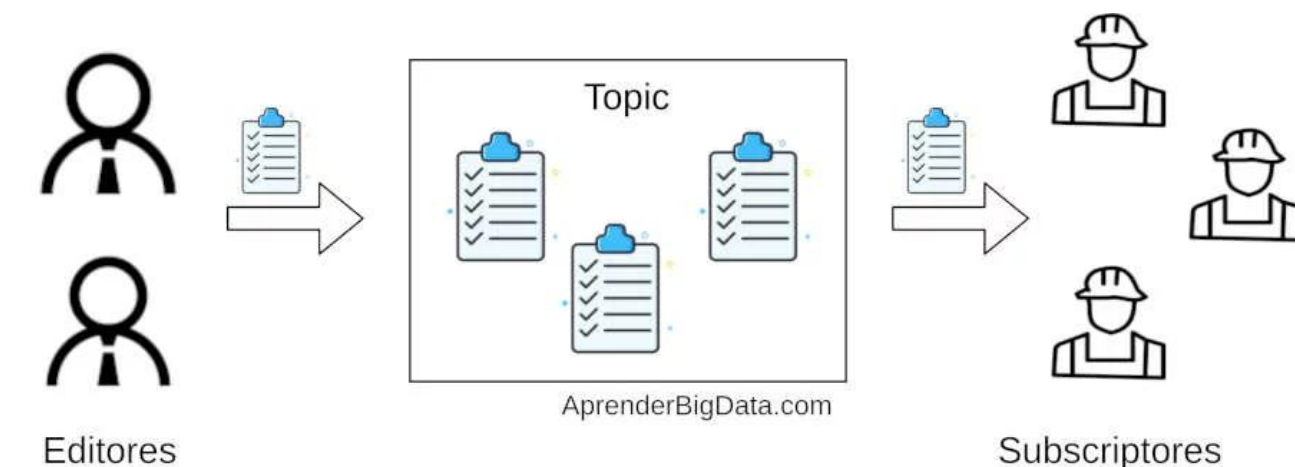
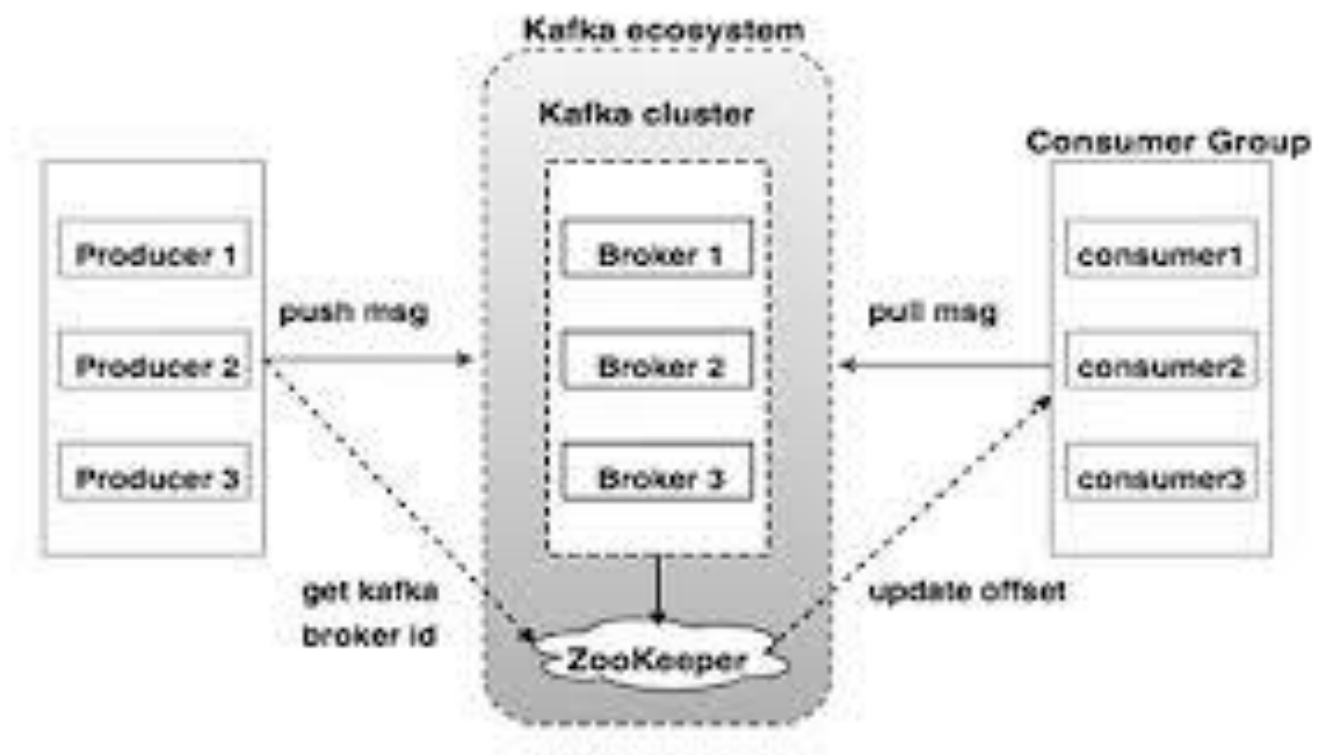
Los **consumers** son aplicaciones que **leen y procesan los mensajes** de Kafka. Se suscriben a uno o varios **topics** y consumen los mensajes en el orden en que fueron publicados. Las características principales son:

- **Consumer Groups:** Los consumidores pueden agruparse en **grupos de consumidores**. Cada grupo de consumidores puede leer mensajes de manera independiente. Kafka distribuye automáticamente las particiones de un topic entre los consumidores dentro de un mismo grupo, lo que permite un procesamiento paralelo.
- **Offsets:** Los consumidores mantienen un registro llamado **offset** que marca el último mensaje leído en una partición. Esto permite que los consumidores reanuden el procesamiento desde donde lo dejaron en caso de un fallo.

c. Brokers (Nodos del Cluster)

Los **brokers** son los servidores que forman el **cluster de Kafka**. Cada broker se encarga de:

- **Almacenar mensajes:** Los brokers almacenan los mensajes de las particiones de los topics. Cada mensaje tiene un **offset** único dentro de la partición que permite a los consumidores leer los mensajes en orden.
- **Gestión de particiones:** Un broker puede ser el líder de una partición, lo que significa que coordina las operaciones de lectura y escritura de esa partición. Los otros brokers mantendrán réplicas de los datos para asegurar la alta disponibilidad.



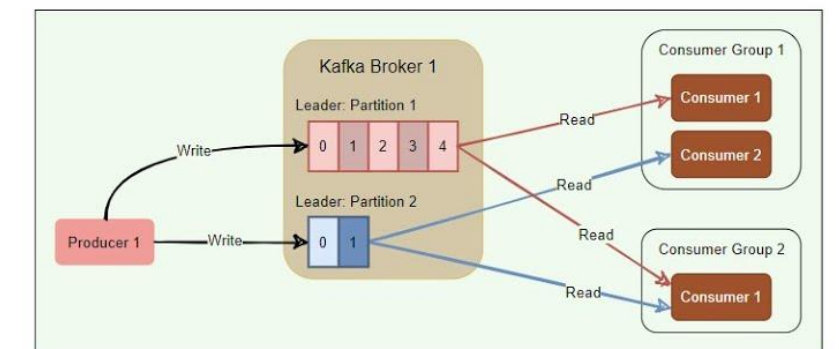
d. Zookeeper (o KRaft en nuevas versiones)

Zookeeper es un sistema de coordinación distribuida que Kafka utilizaba tradicionalmente para tareas de administración del cluster, como:

- **Almacenamiento de metadatos** sobre brokers, topics, particiones y consumidores.
- **Elección del broker líder** para cada partición.
- **Monitoreo de la salud del cluster.**

Sin embargo, en las versiones más recientes de Kafka, se ha empezado a reemplazar Zookeeper con una nueva arquitectura llamada **KRaft**, que es más eficiente y elimina la dependencia de Zookeeper para la coordinación del cluster.

Kafka:- Role of Zookeeper



KRaft (Kafka Raft) es una arquitectura de consenso introducida en Apache Kafka para **eliminar la dependencia de ZooKeeper** para la gestión de metadatos del clúster. Antes de KRaft, Kafka se basaba en ZooKeeper para tareas cruciales como la elección de líderes de partición, la gestión de la pertenencia al clúster y el almacenamiento de la configuración.

¿Cómo funciona KRaft?

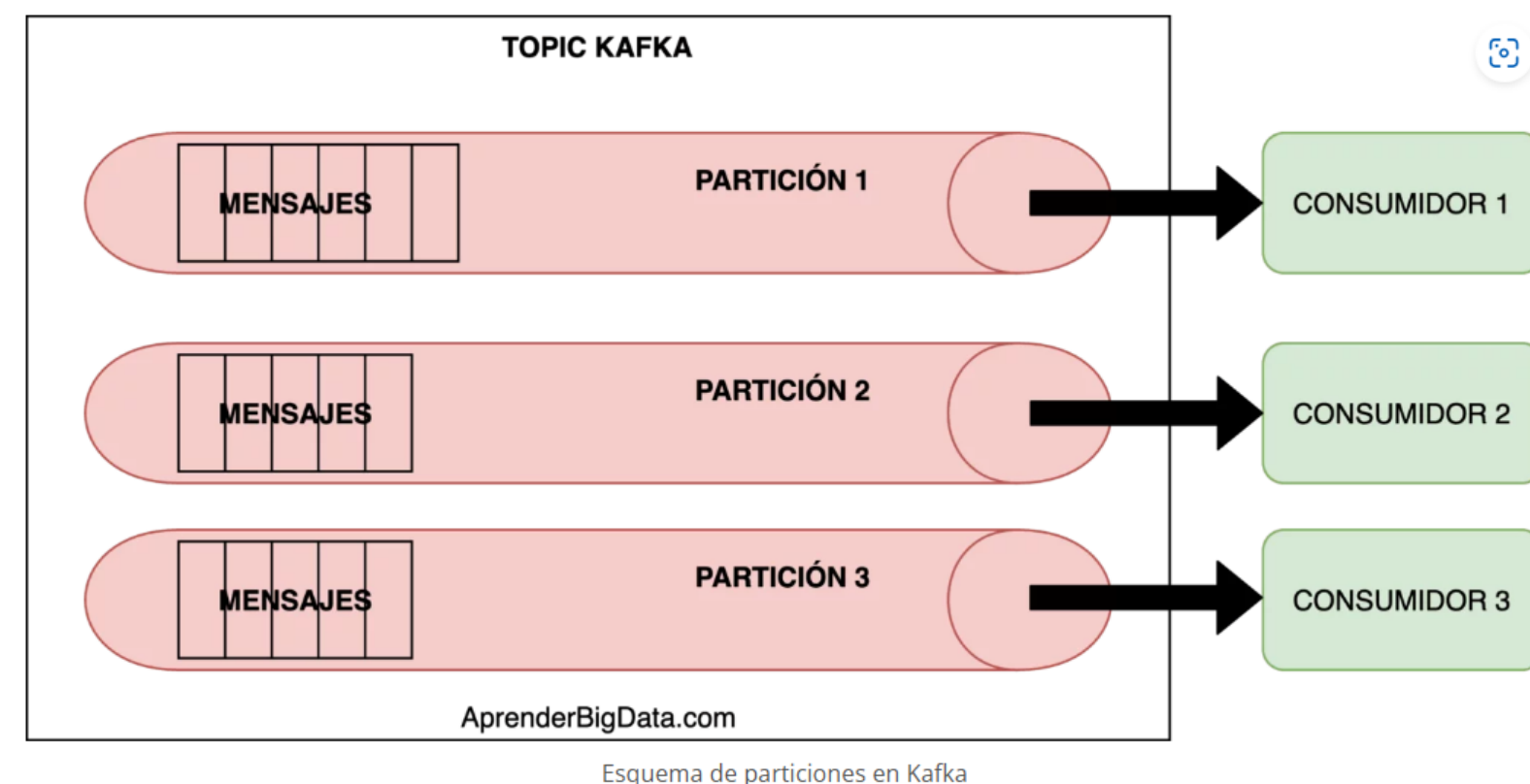
En la arquitectura KRaft, las responsabilidades de gestión de metadatos se integran directamente en los brokers de Kafka. Un clúster de Kafka en modo KRaft opera con un "**quorum de controladores**" (controller quorum). Estos controladores son brokers designados que participan en la elección de un líder y en la replicación del log de metadatos utilizando el protocolo de consenso Raft.

Aquí te presento los componentes clave y su funcionamiento:

- 1.Controladores:** Algunos brokers se configuran para actuar como controladores. Uno de ellos es elegido como el **líder del controlador**, responsable de tomar decisiones sobre los cambios en los metadatos (creación de topics, reasignación de particiones, etc.). Los demás controladores actúan como **seguidores**, replicando el log de metadatos del líder.
- 2.Log de Metadatos:** Todas las modificaciones en los metadatos del clúster se registran en un log que se replica entre los controladores utilizando el protocolo Raft. Esto asegura la consistencia y durabilidad de los metadatos.
- 3.Elección de Líder:** Si el líder del controlador falla, los seguidores inician un proceso de elección para elegir un nuevo líder. El protocolo Raft garantiza que solo un líder sea activo en un momento dado.
- 4.Interacción con los Brokers:** Los brokers regulares (no controladores) se comunican con el líder del controlador para obtener la información más reciente sobre los metadatos del clúster y para coordinar sus acciones.

Un **topic** en Kafka es una categoría o feed específico al que los productores envían mensajes y los consumidores se suscriben. Un topic se subdivide en múltiples **particiones** para mejorar la escalabilidad y permitir la paralelización del procesamiento. Cada partición actúa como un log de datos ordenado:

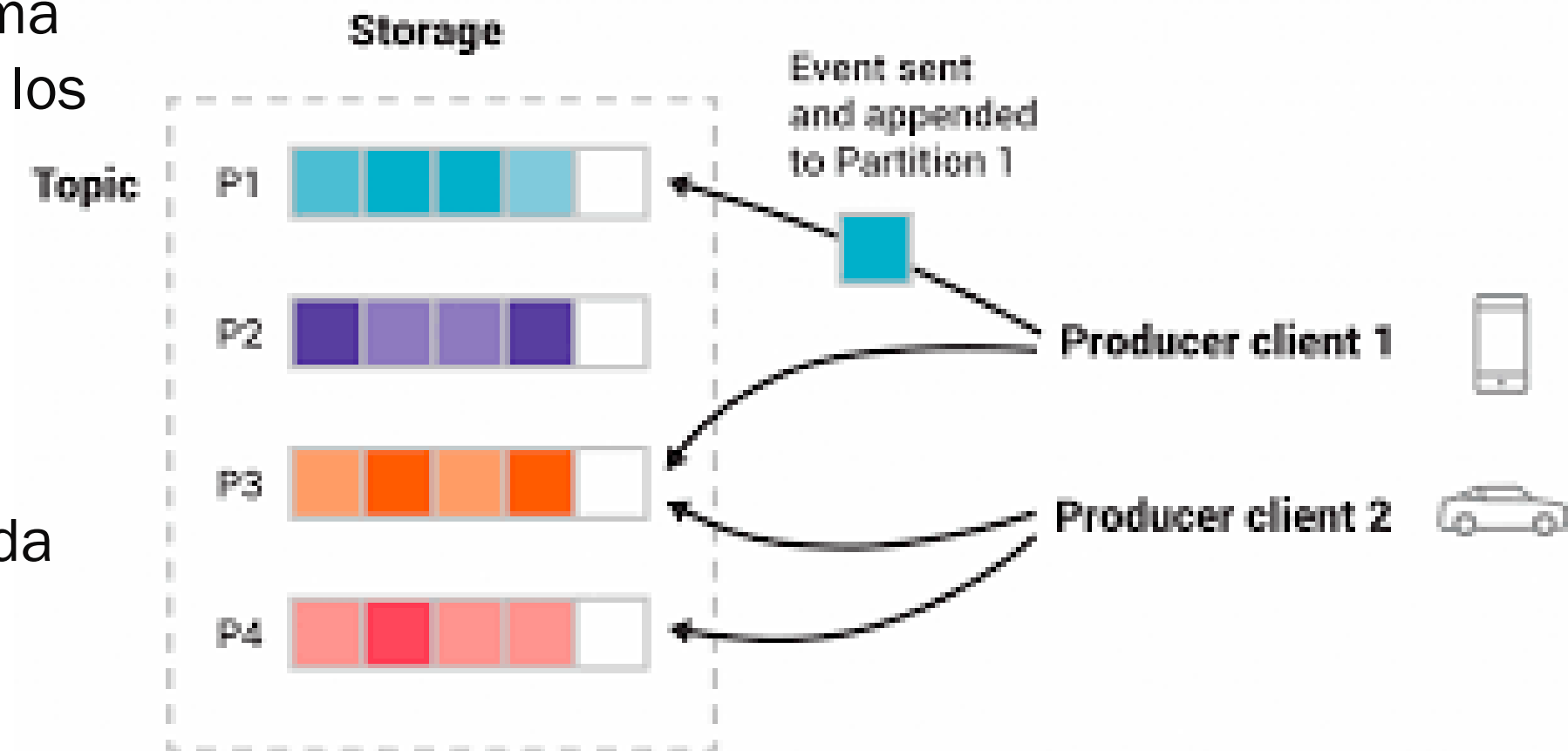
- **Particionamiento:** Cuando un productor envía un mensaje a un topic, Kafka distribuye los mensajes en varias particiones. Este esquema de particionamiento permite que diferentes consumidores lean las particiones en paralelo, mejorando el rendimiento.
- **Particiones como logs ordenados:** Los mensajes dentro de una partición son **ordenados secuencialmente** y cada mensaje tiene un **offset**, que es un identificador único de su posición dentro de la partición. Los consumidores pueden leer los mensajes en el orden en que se almacenaron, y cada consumidor mantiene un registro de hasta dónde ha leído en cada partición utilizando el offset.
- **Réplicas:** Para garantizar la **tolerancia a fallos**, Kafka permite crear múltiples réplicas de una partición en otros brokers. Un broker actúa como el **líder** de una partición, mientras que otros brokers mantienen réplicas de respaldo. Si el broker líder falla, uno de los brokers con réplicas toma el control.



4. Particiones y Réplicas

Cada **topic** en Kafka se divide en particiones, y cada partición se almacena en un broker del cluster. Kafka implementa un sistema de **réplicas** para garantizar la durabilidad y la disponibilidad de los datos. Las particiones y réplicas se organizan de la siguiente manera:

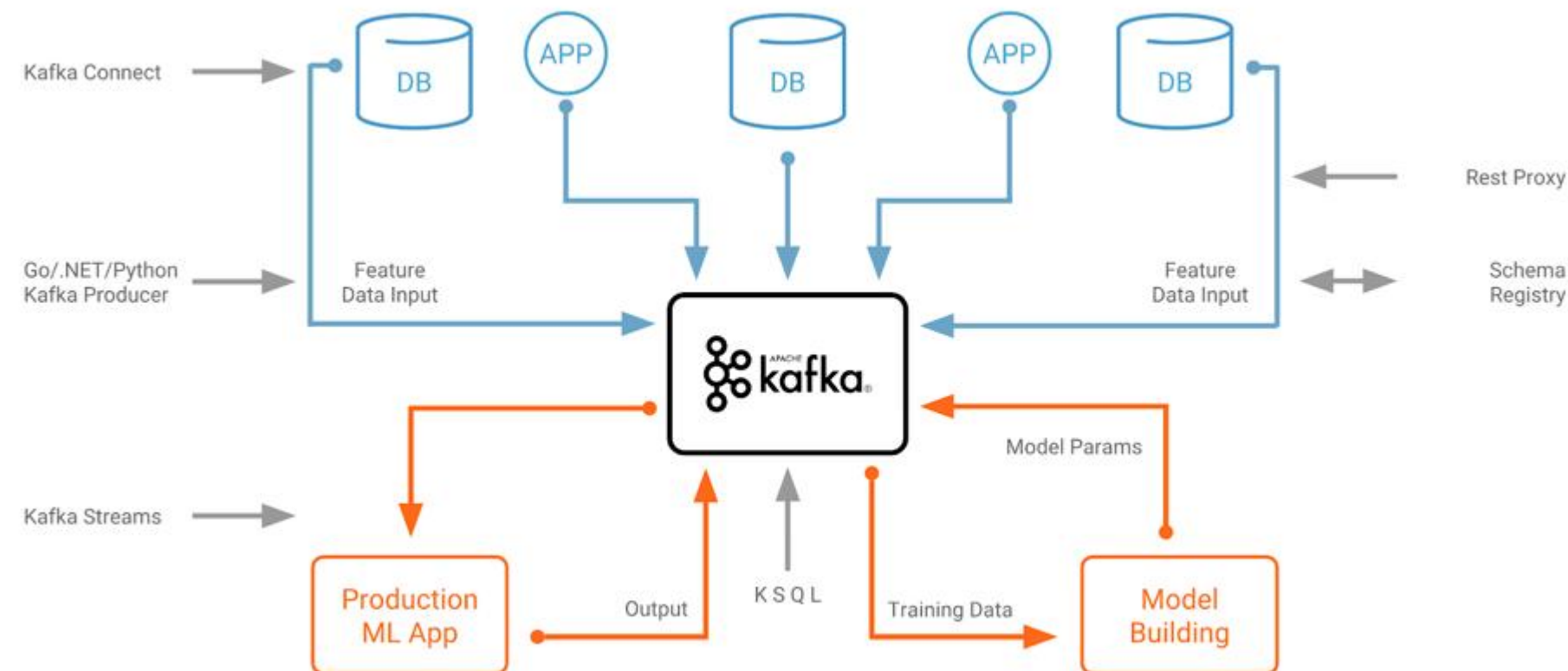
- **Partición líder y réplicas:** Dentro del cluster, cada partición tiene un **líder** y varias **réplicas**. El líder es responsable de manejar todas las operaciones de lectura y escritura de la partición. Las réplicas solo mantienen una copia sincronizada de los datos.
- **Failover automático:** Si el broker líder de una partición falla, uno de los brokers con réplicas asume el rol de líder de forma automática, asegurando la continuidad del servicio sin pérdida de datos.



- **Datos en Tiempo Real para la IA:** La necesidad de procesar datos a medida que se generan para modelos predictivos, sistemas de recomendación, detección de anomalías, etc.

Cómo Kafka Soluciona los Desafíos:

- **Ingestión Masiva de Datos:** Kafka puede manejar grandes volúmenes de datos de múltiples fuentes simultáneamente.
- **Procesamiento en Tiempo Real:** Los consumidores pueden procesar los datos a medida que llegan, sin esperar a que se acumulen grandes lotes.
- **Escalabilidad:** La arquitectura distribuida de Kafka permite escalar horizontalmente añadiendo más brokers y particiones.
- **Tolerancia a Fallos:** La replicación de particiones asegura la disponibilidad de los datos incluso si algunos brokers fallan.
- **Desacoplamiento:** Kafka actúa como un buffer entre los productores de datos y los consumidores (modelos de IA), permitiendo que evolucionen independientemente



Confluent proporciona el aprovechamiento de las ultimas funciones de la IA generativa con una plataforma de streaming de datos. Crea casos prácticos de RAG con datos en tiempo real, contextuales y fiables para tus LLM, bases de datos vectoriales y aplicaciones de GenAI.



Predictive AI

Ensure models are continuously trained on fresh, high-quality data for accurate fraud detection, forecasting, and more. Maintain a consistent view of data across all your systems, data warehouses, and data lakes.

[Learn More >](#)



Generative AI

Build a real-time, contextualized, and trustworthy knowledge base for your GenAI applications. Bring streaming data to LLMs and vector databases to power retrieval-augmented generation (RAG) and prevent hallucinations.

[Learn More >](#)



Agentic AI

Leverage an event-driven architecture to enable dynamic, context-driven workflows for your agents, allowing them to access and share data effortlessly and make decisions based on real-time information.

[Learn More >](#)

INTRODUCCIÓN APACHE KAFKA

 Aprender BIG DATA
AprenderBigData.com

Apache Kafka para principiantes

por Andrés Gómez Ferrer

 **KEEPCODING**
Tech School

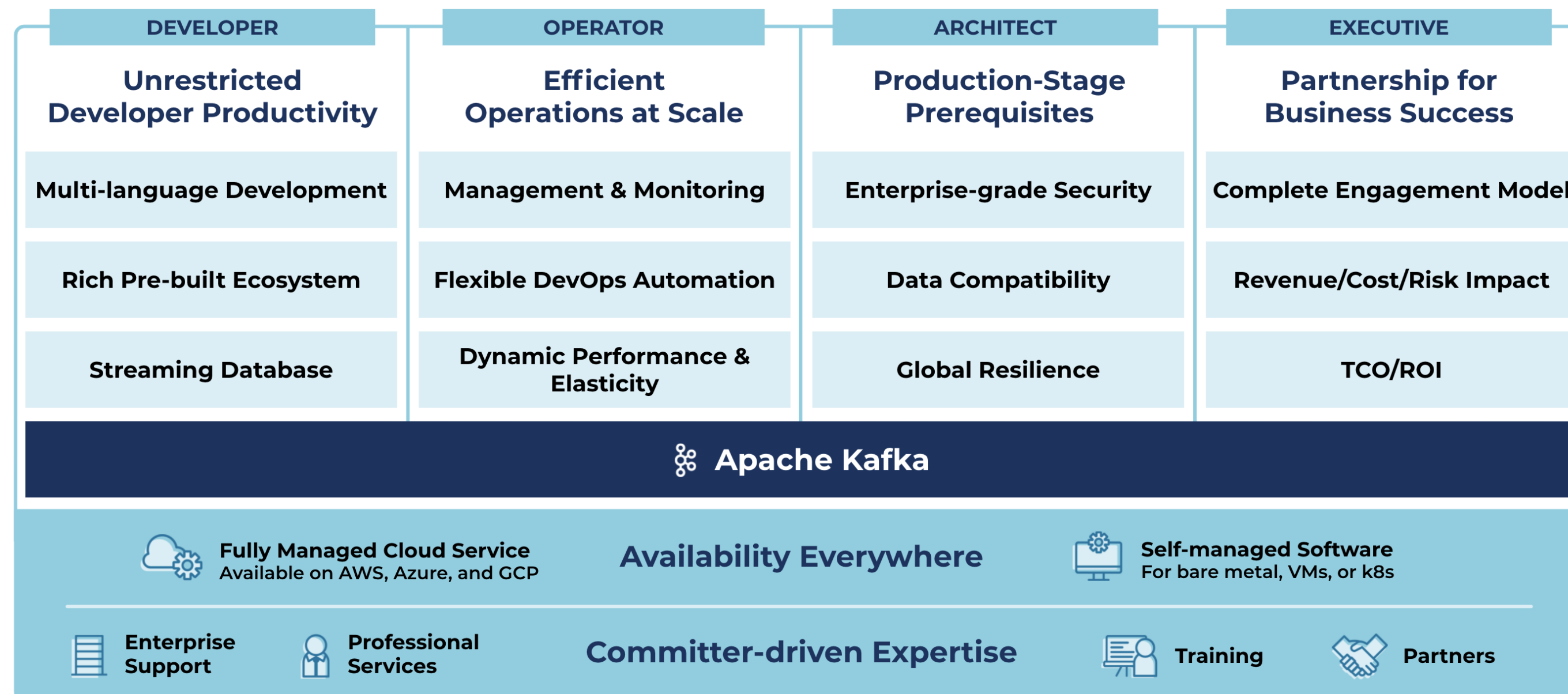


3/ Confluent

¿Qué es Confluent?

Una plataforma completa construida sobre Apache Kafka. Simplifica la gestión, escalabilidad y seguridad de los flujos de datos en tiempo real. Ofrece herramientas y servicios adicionales para el desarrollo y la operación.

¿Por qué Confluent? Acelera la adopción de Kafka en la empresa. Reduce la complejidad operativa. Permite enfocarse en el valor de los datos en tiempo real.



Se trata de una plataforma de streaming basada en Apache Kafka cuyo objetivo es **ayudar a los diferentes roles en la organización en los proyectos de intercambio de información en tiempo real.**

Para ello, desde un punto de vista de arquitectura tecnológica, mejora las capacidades nativas base de Apache Kafka en las siguientes perspectivas:

- Capacidades de Streaming Enterprise
- Conectividad y Gobierno del Dato
- Procesamiento Avanzado
- Monitorización y Gestión
- Seguridad
- Arquitecturas MultiSite y Apoyo en la evolución hacia el Cloud



CONFLUENT

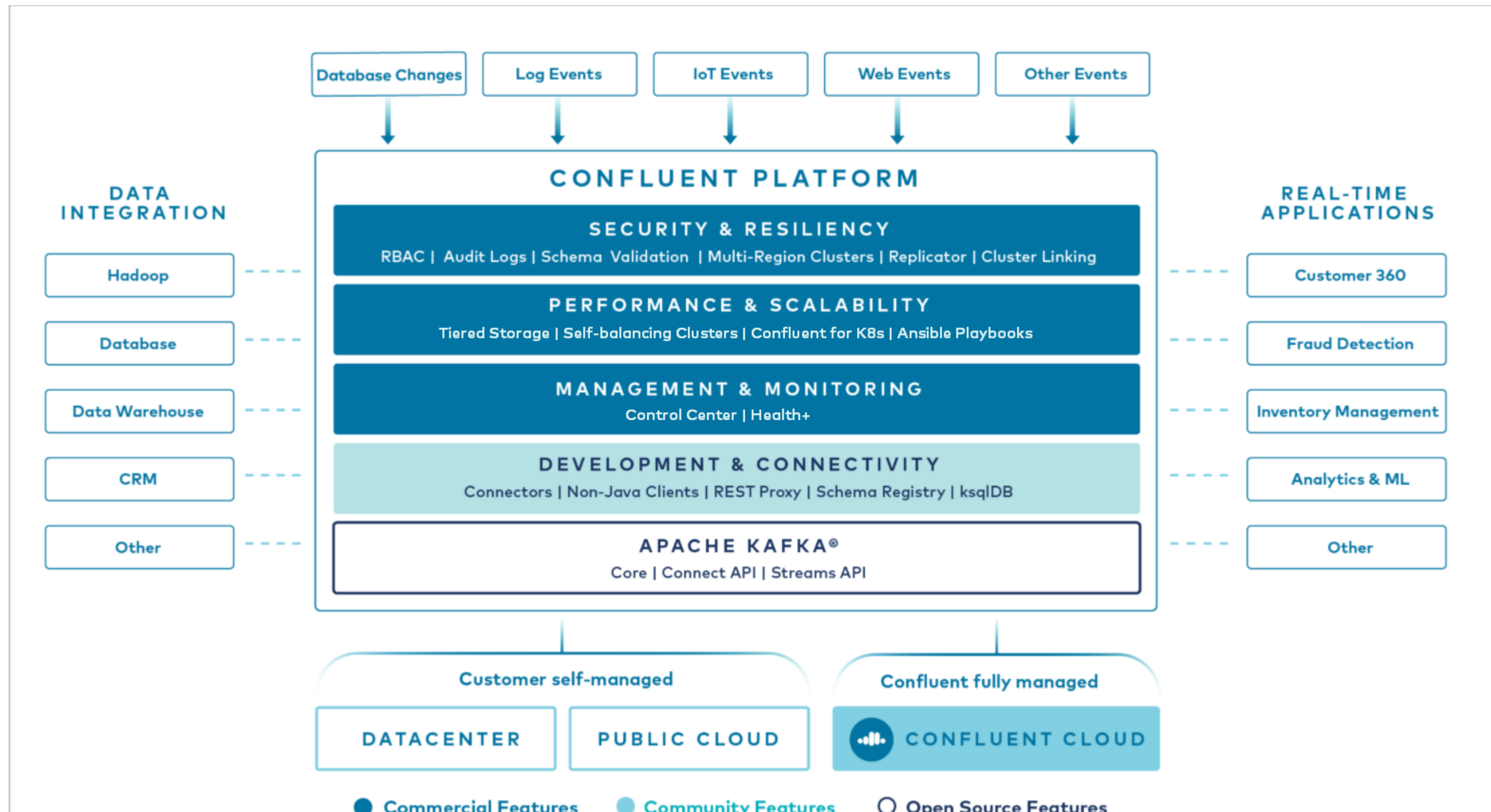
Es capaz de:

- Integrar fuentes productoras y consumidoras de eventos de diferente naturaleza
- Gestionar las capacidades Real Time para transporte de altos volúmenes de datos (Megabytes por segundo)
- Garantizar la resiliencia, escalabilidad y secuencialidad en la transferencia de eventos.
- Colaborar con el resto de los componentes de arquitectura para la gestión de los metadatos, la distribución de carga dentro del clúster.
- Mediante Confluent Server se consiguen maximizar las capacidades de Streaming



Fuente: Confluent

COMPONENTES CONFLUENT



- **Apache Kafka Brokers:**

- El corazón de la plataforma, responsables de recibir, almacenar y servir los streams de eventos (mensajes) organizados en *topics* y *partitions*.
- Diseñados para ser altamente escalables y tolerantes a fallos mediante la replicación.

- **ZooKeeper (o KRaft):**

- **ZooKeeper:** Tradicionalmente utilizado para la gestión del clúster, el seguimiento del estado de los brokers y la elección del líder.
- **KRaft (Kafka Raft):** Una alternativa más reciente que integra las funciones de metadatos directamente en los brokers de Kafka, eliminando la dependencia de ZooKeeper para muchos casos de uso. (Mencionar brevemente si es relevante para tu audiencia).

- **Confluent Schema Registry:**

- Actúa como un registro centralizado para almacenar y gestionar los esquemas de los datos (por ejemplo, Avro, JSON Schema, Protobuf) que fluyen a través de Kafka.
- Asegura que los productores y consumidores de datos utilicen los mismos esquemas, evitando errores de compatibilidad y facilitando la evolución de los datos.

- **ksqlDB Servers:**

- Proporcionan la capa de procesamiento de streams en tiempo real, permitiendo a los usuarios escribir consultas SQL (*ksql*) para transformar, filtrar, agregar y unir streams de datos directamente dentro de Kafka.
- Simplifica la creación de pipelines de procesamiento de datos en tiempo real sin necesidad de escribir código complejo.

- **Confluent Connect Workers:**

- Ejecutan los *conectores* que permiten integrar fácilmente Kafka con sistemas externos, tanto para la ingesta de datos (fuentes) como para la exportación (sinks).
- Soporta una amplia variedad de conectores preconstruidos para bases de datos, sistemas de archivos, APIs, etc.

- **Confluent Control Center:**

- Ofrece una interfaz gráfica intuitiva para monitorizar la salud y el rendimiento del clúster de Confluent, incluyendo brokers, topics, consumidores, conectores y streams de ksqlDB.
- Permite gestionar configuraciones, configurar alertas y obtener información valiosa sobre el flujo de datos.

- **Confluent REST Proxy:** Permite a las aplicaciones interactuar con el clúster de Kafka a través de HTTP.

Énfasis en la Interoperabilidad:

- Resaltar cómo estos componentes trabajan de forma cohesiva para crear una plataforma robusta y completa para la gestión de datos en streaming a gran escala.
- Subrayar que Confluent se basa en la potencia de Apache Kafka y la extiende con funcionalidades clave para la empresa.

Explore nuestros casos de uso más populares

Detección del fraude

Microservicio s event-driven

Pipelines de bases de datos

Integración del mainframe

Pipelines de almacenes de datos

Integración de mensajería

Optimización de SIEM

Análisis de IoT en tiempo real

Reabastecimiento en tiempo real

Redes autónomas

IA generativa

Gestión de riesgos

Análisis en tiempo real

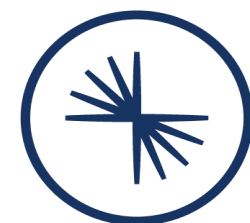
Integración de Security Lake

Análisis predictivo

Inventario en tiempo real

Análisis de vendedores en tiempo real

Pagos en tiempo real



CONFLUENT

Detección de Fraude en Tiempo Real:

- Flujo de datos de transacciones financieras a través de Confluent.
- Modelos de IA desplegados con ksqlDB o integrados a través de Kafka Streams para analizar patrones sospechosos en tiempo real y generar alertas inmediatas.

Personalización de Experiencias de Usuario:

- Captura de la actividad del usuario (clics, compras, visualizaciones) en tiempo real con Confluent.
- Procesamiento con modelos de IA para identificar preferencias y ofrecer recomendaciones personalizadas al instante.

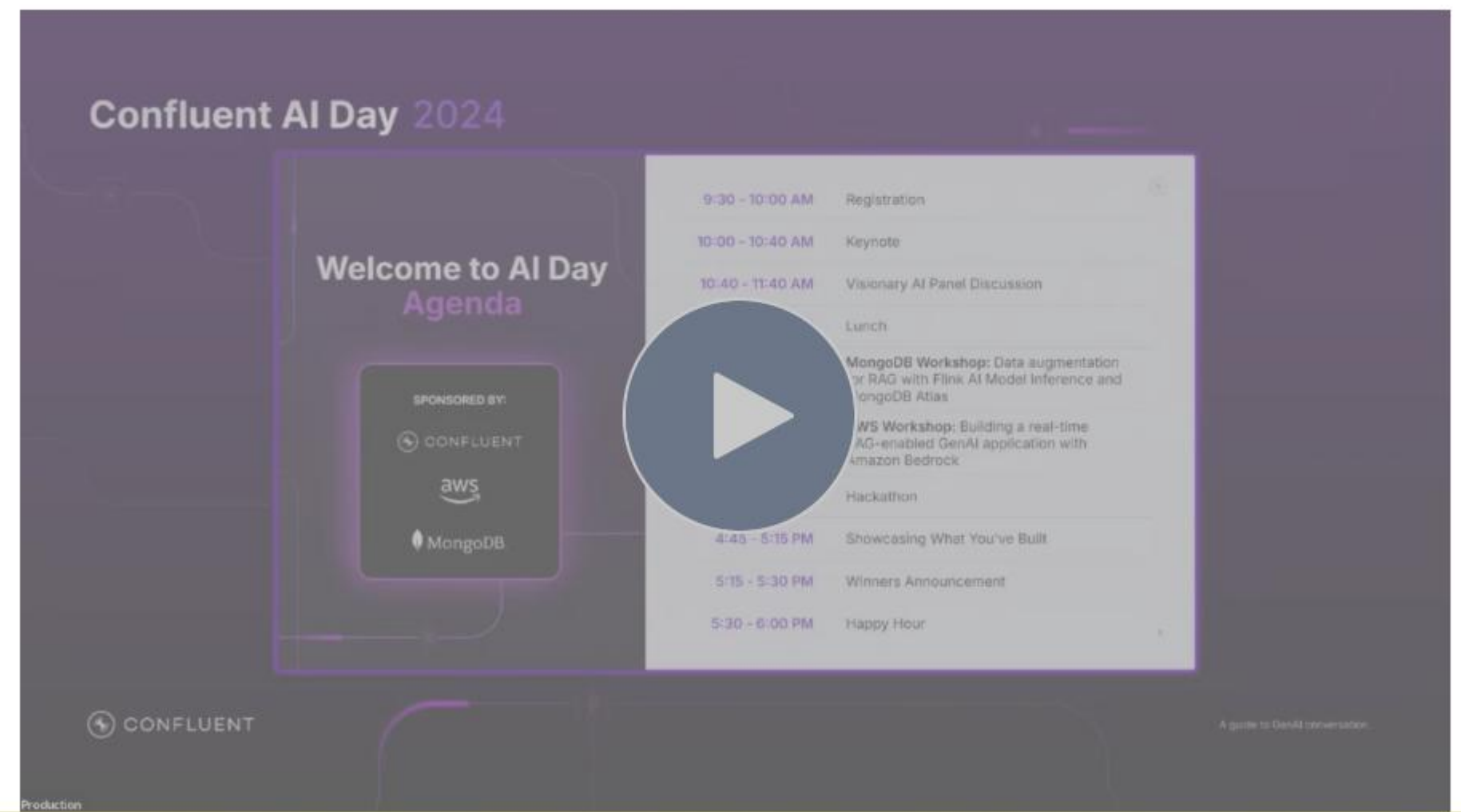
Mantenimiento Predictivo:

- Ingesta de datos de sensores de maquinaria industrial a través de Confluent.
- Análisis con modelos de IA para predecir fallos y programar el mantenimiento antes de que ocurran, minimizando el tiempo de inactividad.

Logística Inteligente:

- Seguimiento en tiempo real de la ubicación y el estado de vehículos y envíos con Confluent.
- Optimización de rutas y predicción de retrasos utilizando modelos de IA para mejorar la eficiencia y la satisfacción del cliente.

Confluent AI Day



Generación Dinámica de Contenido de Marketing Personalizado en Tiempo Real

Escenario: Una empresa de comercio electrónico quiere ofrecer contenido de marketing altamente personalizado a sus usuarios en tiempo real, basándose en su comportamiento de navegación, historial de compras y datos demográficos.

Cómo Confluent facilita este caso de uso:

1.Ingesta de Datos en Tiempo Real: Confluent Connect se utiliza para recopilar datos de diversas fuentes en tiempo real y enviarlos como streams a Apache Kafka:

- 1. Actividad del sitio web:** Clics, visualizaciones de productos, búsquedas, adiciones al carrito.
- 2. Datos de comportamiento:** Historial de compras, valor del pedido, frecuencia de compra.
- 3. Datos de usuario:** Información demográfica (edad, ubicación), preferencias (si se han explicitado), datos de inicio de sesión.

2.Procesamiento y Enriquecimiento de Datos con ksqlDB: ksqlDB se utiliza para procesar y enriquecer estos streams de datos en tiempo real:

- 1. Agregación:** Calcular métricas en tiempo real por usuario (productos vistos recientemente, categorías de interés, gasto promedio en los últimos X días).
- 2. Filtrado:** Seleccionar eventos relevantes para la personalización.
- 3. Unión (Joins):** Combinar datos de diferentes streams (por ejemplo, actividad del sitio web con datos de usuario).
- 4. Creación de "Features" en Tiempo Real:** Generar características relevantes para los modelos de IA Generativa (por ejemplo, "usuario ha visto recientemente productos de la categoría 'electrónica'", "usuario ha comprado previamente productos de la marca 'X'").

3.Integración con Modelos de IA Generativa: Los streams de "features" generados por ksqlDB se envían a una aplicación o servicio que alberga un modelo de IA Generativa. Esto se puede hacer a través de un conector personalizado de Confluent Connect o mediante una aplicación que consume los streams de Kafka utilizando las librerías de cliente de Kafka.

4.Generación de Contenido Personalizado en Tiempo Real: El modelo de IA Generativa recibe las "features" específicas de cada usuario en tiempo real y genera contenido de marketing personalizado:

- 1. Titulares de productos dinámicos:** "Descubre las últimas novedades en [categoría de interés del usuario] que sabemos que te encantarán."
- 2. Descripciones de productos personalizadas:** Resaltando características que son relevantes para el historial o preferencias del usuario.
- 3. Recomendaciones de productos contextuales:** Basadas en lo que el usuario está viendo actualmente y su comportamiento pasado.
- 4. Mensajes de correo electrónico personalizados:** Con líneas de asunto y contenido adaptados a las interacciones recientes del usuario.
- 5. Anuncios dinámicos:** Con creatividades y mensajes que resuenan con los intereses específicos del usuario.

5.Entrega del Contenido Personalizado: El contenido generado se puede enviar de vuelta a través de Kafka a otros sistemas para su visualización en el sitio web, en aplicaciones móviles, en campañas de correo electrónico, o en plataformas de publicidad en tiempo real.

Generación Dinámica de Contenido de Marketing Personalizado en Tiempo Real

Característica	Apache Kafka (Open Source)	Confluent Platform
Gestión de Esquemas	No integrado (requiere soluciones externas)	Confluent Schema Registry: Integrado y robusto
Procesamiento Stream	Kafka Streams: Librería de bajo nivel	ksqlDB: Streaming SQL de alto nivel, simplifica el desarrollo
Conectores de Datos	Kafka Connect: Framework básico, comunidad de conectores	Confluent Connect: Amplia gama de conectores preconstruidos y soporte
Monitorización	Herramientas de terceros	Confluent Control Center: Monitorización y gestión centralizada
Seguridad	Configuración manual y compleja	Funciones de seguridad avanzadas y simplificadas
Escalabilidad	Requiere experiencia para una escalabilidad óptima	Herramientas y servicios para una escalabilidad más sencilla
Soporte	Soporte de la comunidad	Soporte comercial de Confluent
Coste	Gratuito (costes de infraestructura y gestión)	Modelo de suscripción (basado en el uso)

Caso de Uso: Agente Conversacional Inteligente para Soporte al Cliente con RAG en Tiempo Real

Escenario: Una gran empresa de telecomunicaciones busca mejorar su servicio de atención al cliente proporcionando respuestas más precisas y contextuales a las preguntas de sus usuarios a través de un agente conversacional inteligente. La información relevante para responder a estas preguntas reside en diversas fuentes de conocimiento, como bases de datos de productos, documentación técnica, artículos de preguntas frecuentes (FAQ) y el historial de interacciones con los clientes.

Cómo Confluent facilita este caso de uso con RAG:

1. Ingesta de Datos en Tiempo Real de Fuentes de Conocimiento: Confluent Connect se utiliza para ingerir datos en tiempo real (o de forma continua) desde las diversas fuentes de conocimiento de la empresa hacia Apache Kafka:

1. **Bases de datos de productos:** Información actualizada sobre planes, precios, características.
2. **Sistemas de gestión de documentación:** Artículos técnicos, manuales de usuario.
3. **Bases de datos de FAQs:** Preguntas y respuestas comunes.
4. **Historial de interacciones con clientes:** Registros de chats, llamadas y correos electrónicos anteriores (anonimizados y agregados).

2. Procesamiento y Vectorización de Datos con ksqlDB y/o Aplicaciones de Streaming:

1. **ksqlDB:** Se puede utilizar para realizar transformaciones iniciales en los datos (por ejemplo, extraer texto relevante).
2. **Aplicaciones de Streaming (Kafka Streams o Flink integrados con Confluent):** Estas aplicaciones consumen los streams de datos de Kafka y realizan las siguientes tareas:
 1. **Generación de Embeddings (Vectores):** Utilizan modelos de lenguaje (embeddings models) para convertir el contenido textual de las fuentes de conocimiento en representaciones vectoriales densas. Estos vectores capturan el significado semántico del texto.
 2. **Almacenamiento de Vectores:** Los vectores generados se almacenan en una base de datos vectorial especializada (como Pinecone, Weaviate o Milvus). Junto con el vector, se almacena una referencia al documento original en Kafka o en el sistema fuente.

3. Interacción del Usuario con el Agente Conversacional:

1. El usuario interactúa con el agente conversacional a través de una interfaz de chat.
2. La pregunta del usuario se envía a un servicio de procesamiento del lenguaje natural (NLP).

Generación Dinámica de Contenido de Marketing Personalizado en Tiempo Real

1. Recuperación de Información Relevante (RAG):

1. El servicio NLP genera un vector de embedding para la pregunta del usuario.
2. Este vector de consulta se utiliza para buscar en la base de datos vectorial los documentos más similares (semánticamente relacionados) de las fuentes de conocimiento.
3. Se recuperan los fragmentos de texto relevantes (junto con sus referencias a Kafka o la fuente original).

2. Aumento de la Generación con el Contexto Recuperado:

1. La pregunta original del usuario y los fragmentos de texto relevantes recuperados se envían a un modelo de lenguaje grande (LLM) para generar una respuesta.
2. El LLM utiliza el contexto proporcionado por los fragmentos recuperados para ofrecer una respuesta más precisa, informativa y contextualizada.

3. Entrega de la Respuesta al Usuario: La respuesta generada por el LLM se muestra al usuario a través de la interfaz del agente conversacional.

4. Registro de Interacciones y Feedback (Opcional): Las preguntas de los usuarios, los fragmentos de conocimiento recuperados y las respuestas generadas pueden enviarse de vuelta a Kafka para su análisis, mejora del sistema y posible re-entrenamiento de los modelos.

Cómo Confluent facilita el RAG en tiempo real:

- **Flujo de Datos Continuo:** Confluent asegura un flujo constante y en tiempo real de las actualizaciones de las fuentes de conocimiento, lo que permite que el agente conversacional tenga acceso a la información más reciente.
- **Baja Latencia:** La arquitectura de Kafka está diseñada para el procesamiento de datos de baja latencia, lo que es crucial para una experiencia de usuario fluida y en tiempo real con el agente conversacional.
- **Escalabilidad:** Confluent puede manejar grandes volúmenes de datos de las fuentes de conocimiento y un gran número de interacciones de usuarios simultáneas.
- **Integración Sencilla:** Confluent Connect facilita la conexión con diversas bases de datos y sistemas de almacenamiento de conocimiento.
- **Procesamiento en Tiempo Real:** ksqlDB y las aplicaciones de streaming permiten realizar transformaciones y la generación de embeddings de manera eficiente a medida que los datos llegan.

).

3/ BBDD NoSQL

El término **NoSQL** significa "Not Only SQL" o "No SQL", refiriéndose a un conjunto de sistemas de gestión de bases de datos que no siguen el modelo tradicional de bases de datos relacionales basadas en SQL. Surgieron como respuesta a las limitaciones de escalabilidad y flexibilidad de las bases de datos relacionales, especialmente cuando las empresas empezaron a enfrentarse a volúmenes masivos de datos, como en el caso de redes sociales, comercio electrónico, y análisis en tiempo real.

Características principales de NoSQL:

- **Esquemas flexibles:** A diferencia de las bases de datos SQL, las bases de datos NoSQL no requieren un esquema fijo. Los datos pueden tener diferentes estructuras dentro de la misma colección o tabla.
- **Distribución horizontal:** Permiten la partición de datos (sharding) en múltiples servidores para mejorar el rendimiento y la escalabilidad.
- **Alta disponibilidad y tolerancia a fallos:** Están diseñadas para entornos distribuidos, lo que las hace más resistentes a fallos de hardware o red.



Las bases de datos NoSQL surgieron como una solución para manejar las necesidades de las aplicaciones modernas, que se enfrentan a un gran volumen, variedad y velocidad de datos (las 3 V's del Big Data). Algunas de las necesidades clave que resuelven son:

- **Escalabilidad horizontal:** Las bases de datos NoSQL permiten distribuir los datos en múltiples nodos de forma eficiente, algo que las bases de datos relacionales no hacen tan bien. Esto es fundamental en aplicaciones como redes sociales, donde la cantidad de usuarios y datos crece exponencialmente.
- **Flexibilidad de almacenamiento:** Muchas aplicaciones actuales manejan datos no estructurados o semi-estructurados, como JSON, XML o archivos multimedia. Las bases de datos NoSQL permiten almacenar estos datos sin la necesidad de un esquema fijo, lo que reduce la complejidad.
- **Rendimiento en tiempo real:** Las aplicaciones como los sistemas de recomendación, el análisis en tiempo real o la publicidad digital requieren tiempos de respuesta muy rápidos con grandes volúmenes de datos. Las bases de datos NoSQL están optimizadas para estas operaciones rápidas de lectura y escritura.

Ventajas de las bases de datos NoSQL

Flexibilidad

Las bases de datos NoSQL pueden almacenar y administrar datos en varios formatos, incluidos JSON, XML, pares clave-valor y más. Esta flexibilidad permite a las organizaciones adaptarse a los requisitos de datos cambiantes sin alterar el esquema de su base de datos.

Escalabilidad

Las bases de datos NoSQL están hechas para crecer de forma horizontal. Pueden trabajar con mucha información y mucha gente repartiendo los datos entre varios puntos o grupos, lo que asegura que la velocidad no se pierda a medida que hay más demanda.

Alta disponibilidad

Tienen sistemas incorporados para asegurar que no se caigan, ni siquiera cuando hay problemas con los equipos o la conexión. Esto es muy importante para aplicaciones que no pueden parar, que necesitan estar funcionando siempre.

Funcionalidad mejorada

Las bases de datos NoSQL tienen tipos de datos que se adaptan a cada situación, que nos ofrecen modelos de datos distintos y formas de trabajar con ellos muy sencillas.

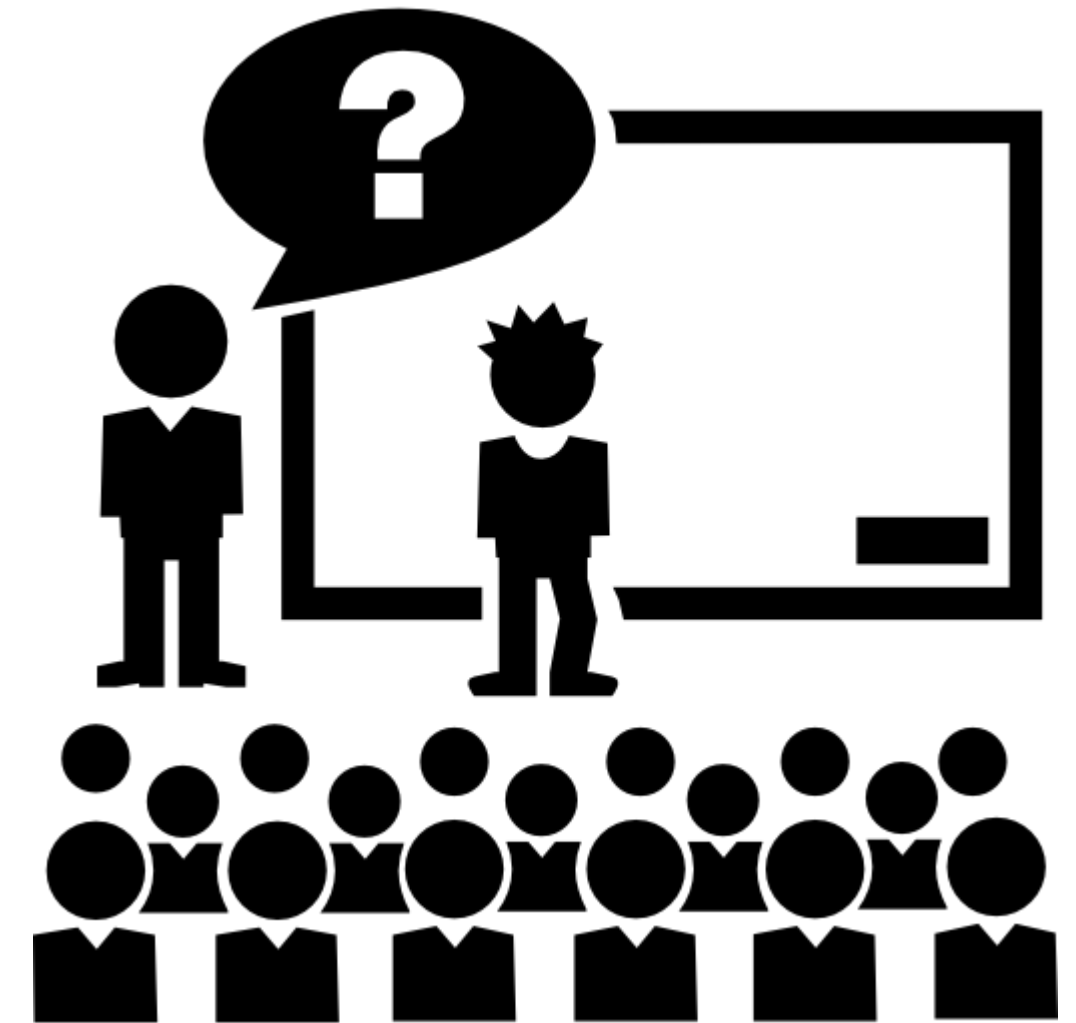
SQL (Structured Query Language) ha sido la base de los sistemas de gestión de bases de datos relacionales (RDBMS) durante décadas, proporcionando un modelo estructurado y estandarizado para manejar datos.

- **Estructura de datos:**
 - **SQL:** Basadas en tablas con filas y columnas, lo que requiere que todos los datos sigan un esquema predefinido.
 - **NoSQL:** Pueden almacenar datos no estructurados o semi-estructurados (JSON, documentos, pares clave-valor), lo que las hace más flexibles.
- **Escalabilidad:**
 - **SQL:** Escala verticalmente (añadiendo más recursos a un solo servidor).
 - **NoSQL:** Escala horizontalmente (añadiendo más servidores), lo que es ideal para aplicaciones con un gran volumen de datos.
- **Consistencia vs Disponibilidad:** Introducir el **Teorema CAP** (Consistencia, Disponibilidad y Tolerancia a particiones).
 - **SQL:** En su mayoría, priorizan la **consistencia fuerte** (todos los usuarios ven la misma versión de los datos en cualquier momento).
 - **NoSQL:** A menudo optan por **disponibilidad y tolerancia a particiones** en sistemas distribuidos, aceptando cierto grado de inconsistencia temporal (consistencia eventual).

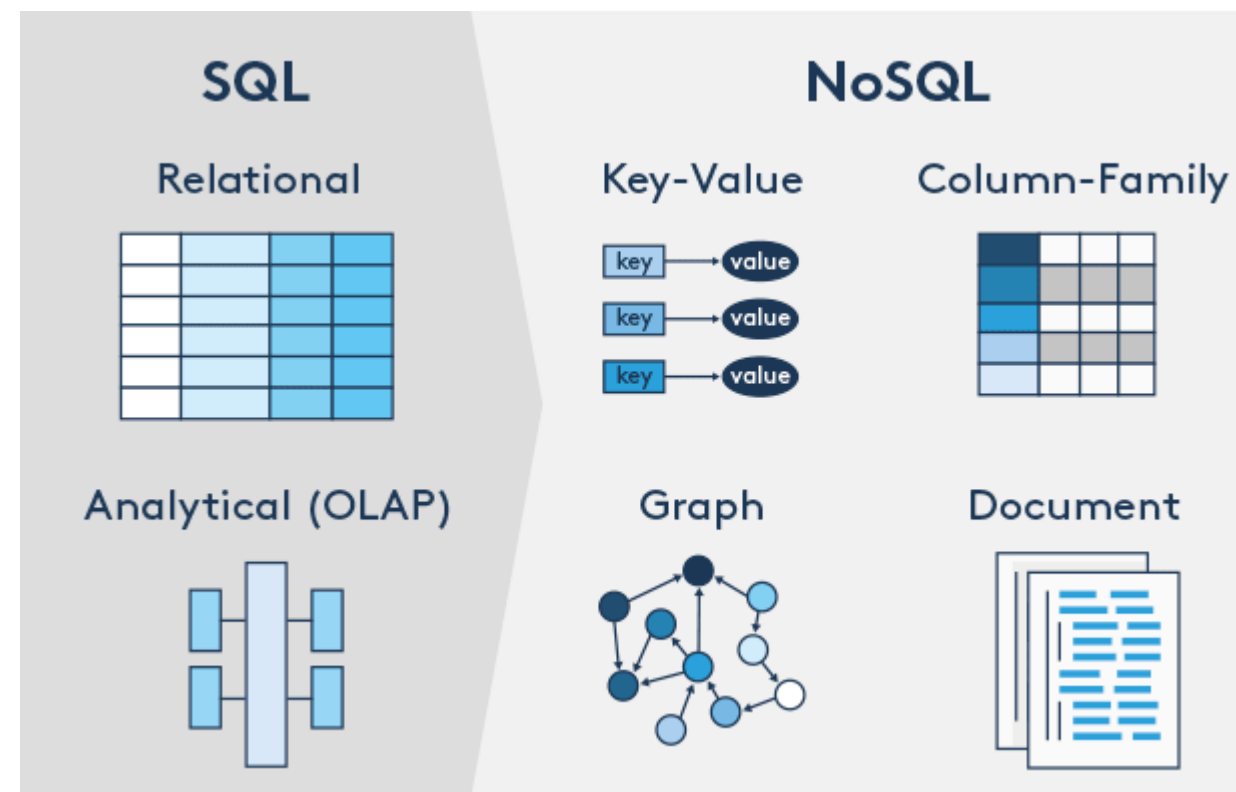
Característica	SQL	NoSQL
Estructura de datos	Esquema fijo (tablas)	Esquema flexible
Consistencia	Consistencia fuerte	Consistencia eventual
Escalabilidad	Vertical	Horizontal
Tipo de transacciones	ACID (transacciones completas)	BASE (transacciones más ligeras)
Aplicaciones típicas	ERP, CRM, contabilidad	Redes sociales, Big Data

Pregunta a la clase: "¿Cuáles de las aplicaciones que utilizan diariamente creen que usan NoSQL?"

(Ejemplos: redes sociales como Facebook o Twitter, aplicaciones de mensajería como WhatsApp, plataformas de streaming como Netflix, AEAT)".



En el ecosistema de bases de datos NoSQL, existen cuatro tipos principales, cada uno de ellos diseñado para casos de uso específicos según la estructura y las necesidades del tipo de datos que manejan. Estos son: bases de datos de **clave-valor**, **documentos**, **columnar** y **grafos**.



BBDD Clave-Valor

Una base de datos clave-valor es el modelo más simple de base de datos NoSQL. En este tipo de bases de datos, los datos se almacenan como pares **clave-valor**, donde una clave única identifica a un valor asociado. Este modelo es muy eficiente para operaciones de lectura y escritura rápidas, lo que lo hace ideal para caché y sistemas de alto rendimiento.

Funcionamiento: El acceso a los datos se realiza a través de una clave única, como si fuera un gran diccionario o mapa en programación.

Casos de uso:

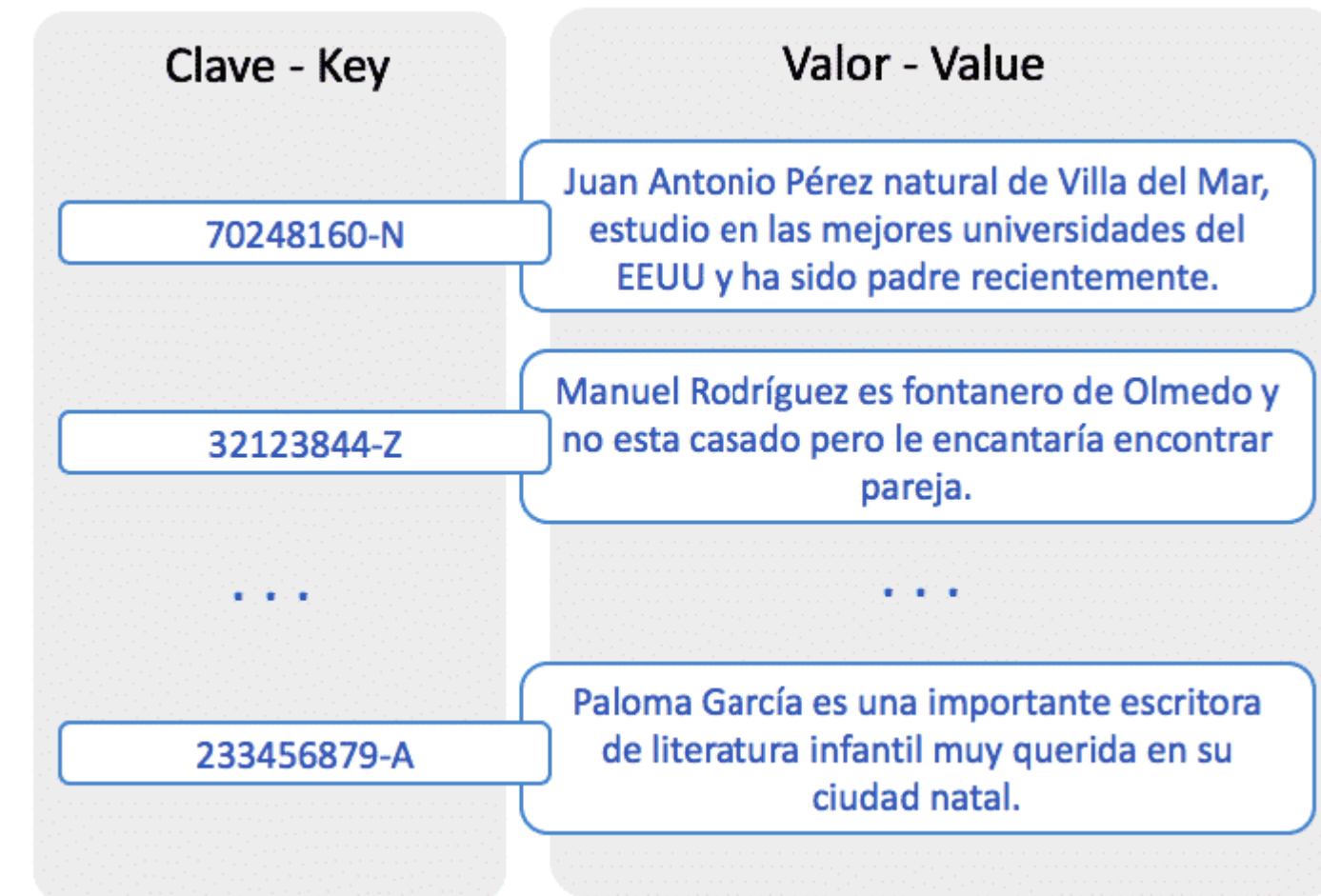
Sistemas de caché: Donde se almacenan resultados de consultas costosas para acceso rápido (ej. Redis).

Sesiones de usuario: Donde se guarda el estado del usuario en aplicaciones web.

Ejemplos:

Redis: Un sistema de base de datos en memoria que se usa principalmente para almacenar estructuras de datos en caché.

Amazon DynamoDB: Una base de datos NoSQL administrada que permite una escalabilidad masiva con baja latencia.



json

Copiar código

```
{
  "user123": "John Doe",
  "session567": "active",
  "cart789": "item1, item2, item3"
}
```

Las bases de datos de documentos almacenan los datos en formato de documentos, habitualmente en **JSON**, **BSON** o **XML**. Cada documento es una entidad autónoma que puede tener un esquema flexible, lo que significa que los diferentes documentos en una misma colección no necesitan tener el mismo formato.

- **Funcionamiento:** Los datos se organizan en documentos, cada uno de los cuales puede tener diferentes campos. Los documentos se agrupan en colecciones y se pueden indexar para facilitar consultas rápidas.

Casos de uso:

- **Aplicaciones web:** Que manejan grandes cantidades de datos semiestructurados, como perfiles de usuario, registros de actividad, o información de productos.
- **Sistemas de gestión de contenido (CMS):** Donde los datos tienen estructura flexible o cambiante.

Ejemplos:

- **MongoDB:** Es el sistema de base de datos de documentos más popular, ideal para aplicaciones que requieren alta flexibilidad en el manejo de datos.
- **CouchDB:** Utiliza JSON para almacenar datos y ofrece una API HTTP para realizar consultas.

Visualización (ejemplo de MongoDB):

```
json Copiar código
{
  "_id": "603c9c2b2f1f4f07a44f05a9",
  "name": "John Doe",
  "email": "johndoe@example.com",
  "orders": [
    { "order_id": "001", "total": 100, "status": "shipped" },
    { "order_id": "002", "total": 200, "status": "pending" }
  ]
}
```

Las bases de datos columnar, también conocidas como **almacenes de columnas**, almacenan los datos en columnas en lugar de filas. Este tipo de bases de datos es eficiente para realizar consultas en grandes volúmenes de datos y se usa comúnmente en aplicaciones analíticas y de Big Data, donde las consultas requieren leer muchas columnas a la vez.

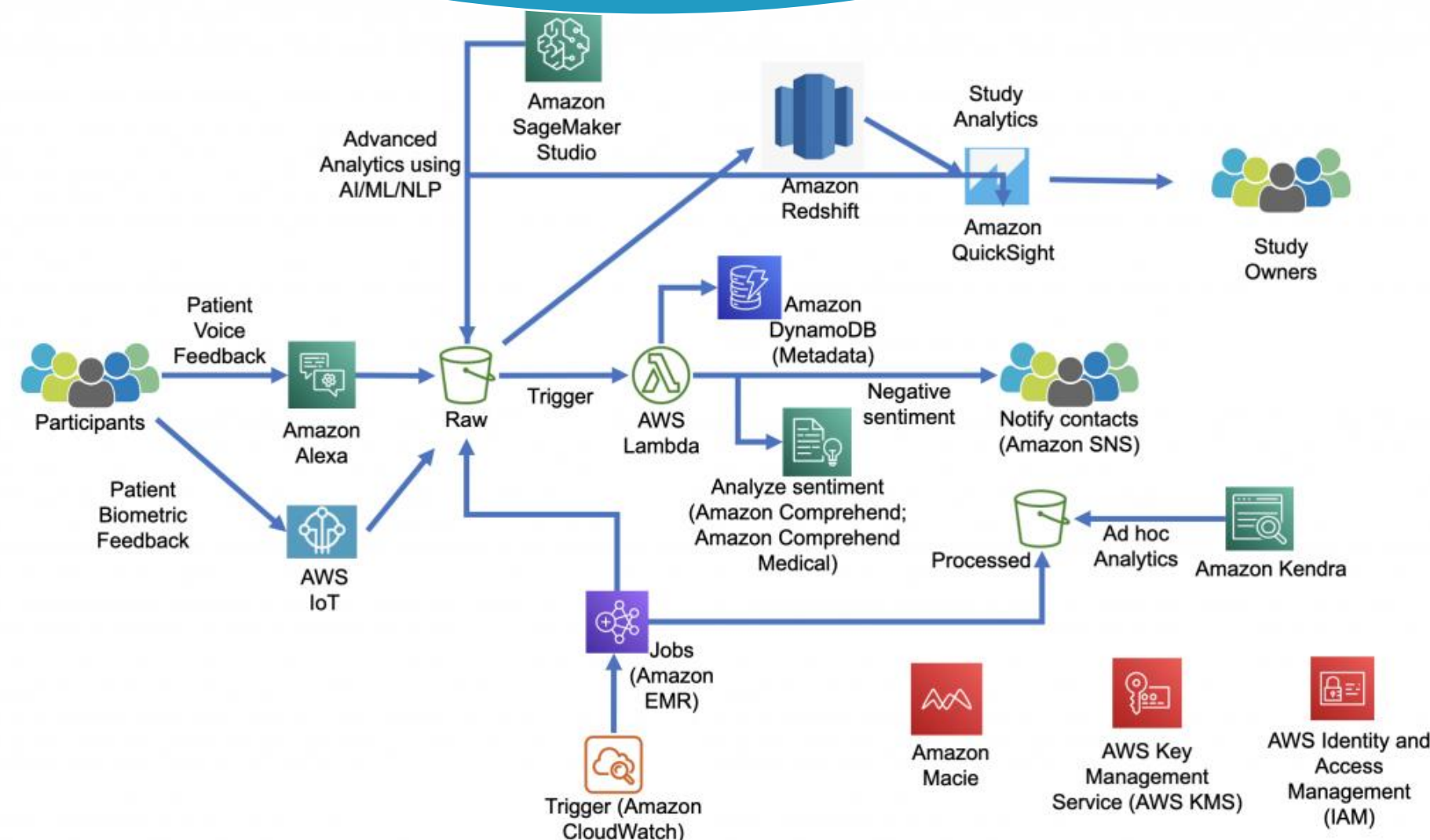
Funcionamiento: Los datos se agrupan por columnas en lugar de por filas, lo que permite leer solo las columnas necesarias para una consulta, mejorando el rendimiento en operaciones de lectura intensiva.

Casos de uso:

- **Analítica y Big Data:** En sistemas que manejan grandes volúmenes de datos y requieren realizar análisis rápido de ciertos campos (ej. **Hadoop HBase**).
- **Sistemas de recomendación:** Donde es necesario procesar grandes volúmenes de datos y buscar correlaciones rápidamente.

Ejemplos:

- **Apache Cassandra:** Optimizada para manejar grandes cantidades de datos a través de muchos servidores.
- **HBase:** Una base de datos distribuida construida sobre Hadoop
- **Amazon Redshift** es un servicio de **almacenamiento de datos en la nube** diseñado y administrado por Amazon Web Services (AWS).



Las bases de datos de grafos están diseñadas para almacenar relaciones entre entidades. Los datos se organizan en nodos (entidades) y aristas (relaciones), y este enfoque es ideal para representar y consultar redes complejas, como redes sociales o mapas de tráfico.

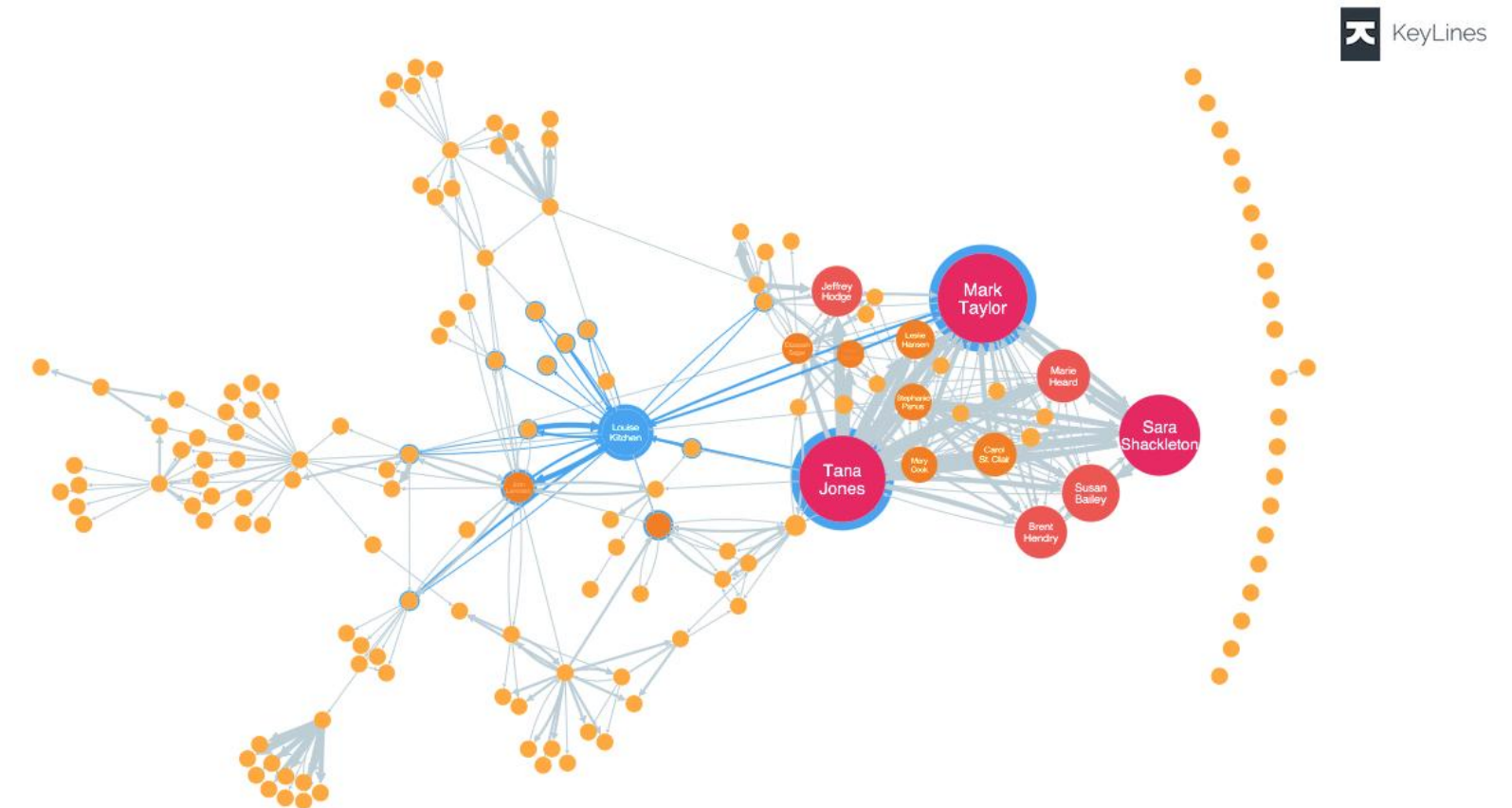
Funcionamiento: Utilizan un modelo de grafos, donde los nodos representan entidades (personas, lugares, objetos) y las aristas representan relaciones entre ellas. Esto permite realizar consultas complejas sobre las relaciones de los datos de manera eficiente.

Casos de uso:

- **Redes sociales:** Donde las relaciones entre personas (amistades, seguidores, etc.) son fundamentales.
- **Recomendaciones:** Como en los sistemas de recomendación de productos o amigos, que necesitan encontrar patrones en las relaciones.
- **Mapas y rutas:** Para aplicaciones que necesitan realizar cálculos complejos de rutas o conectividad.

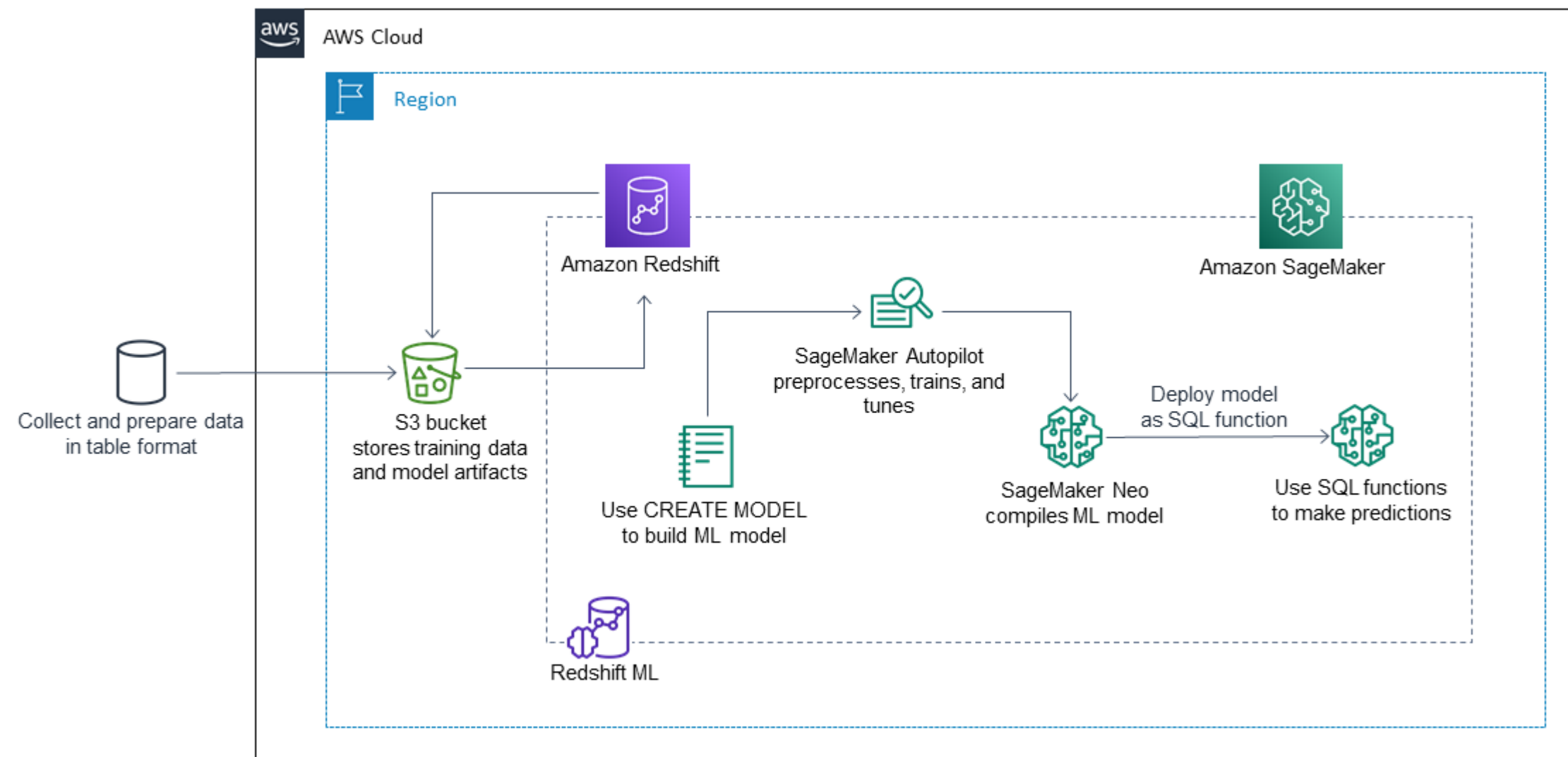
Ejemplos:

- **Neo4j:** Es una de las bases de datos de grafos más utilizadas, popular por su capacidad para realizar consultas sobre redes complejas.
- **Amazon Neptune:** Un servicio de base de datos de grafos completamente gestionado.

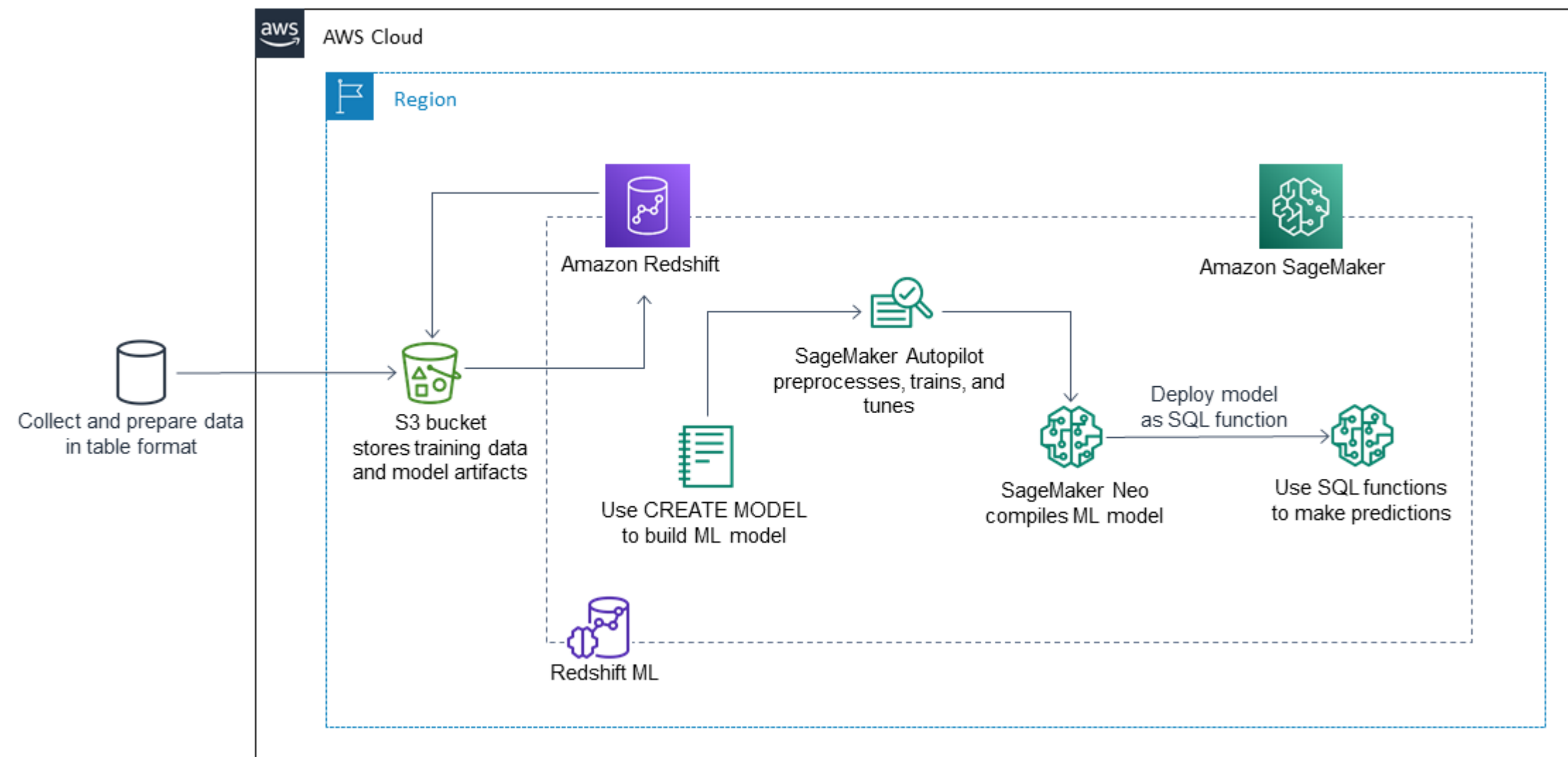


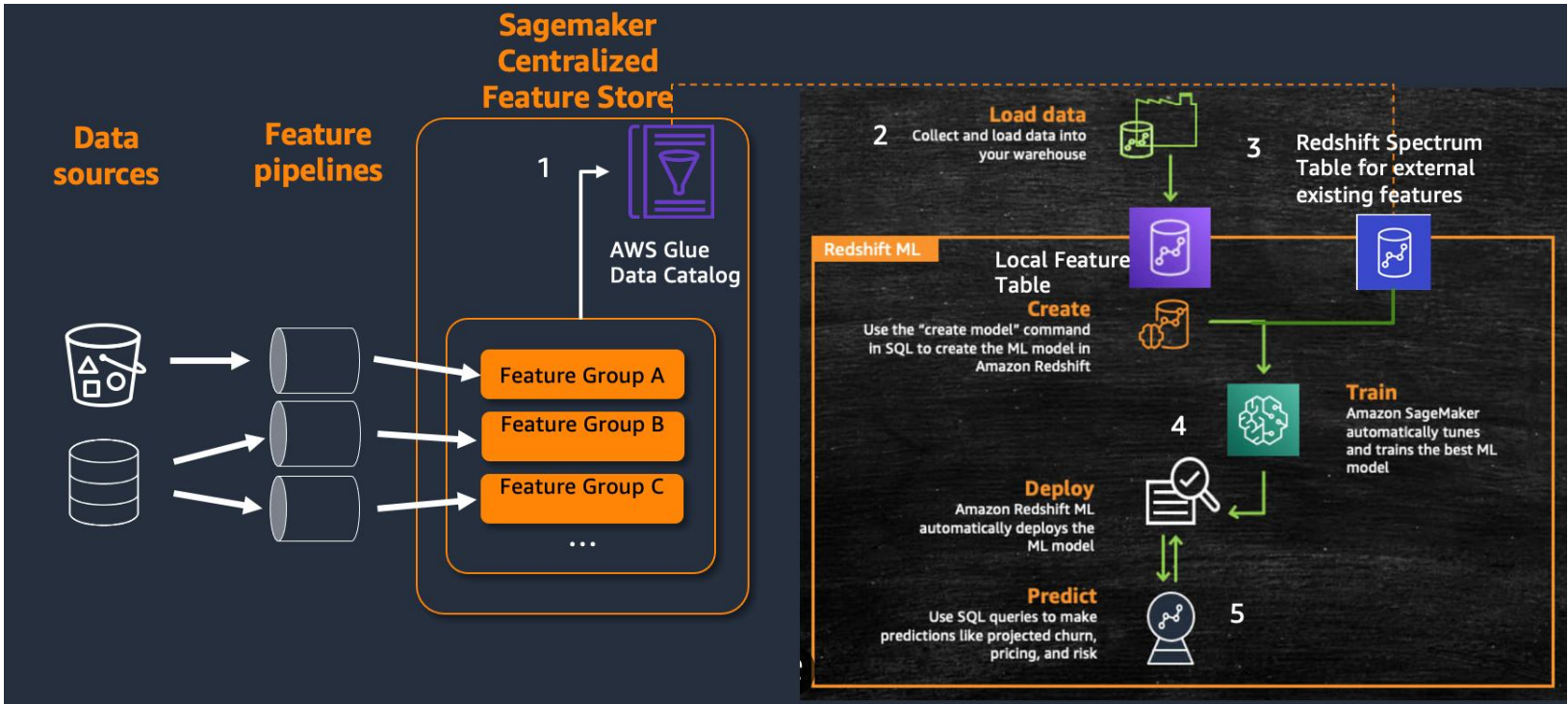
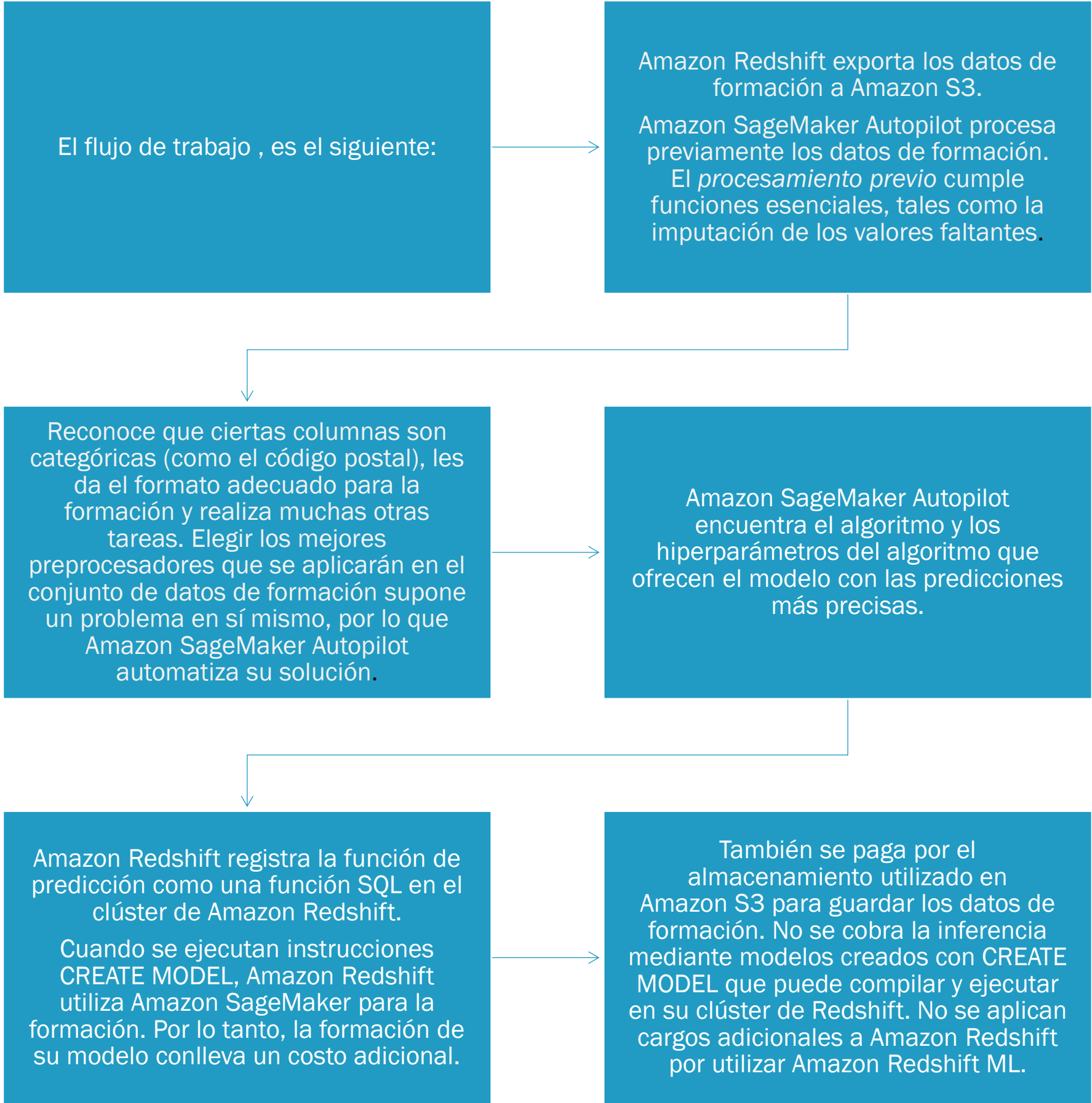
4/ Redshift

Crea, entrene e implemente modelos de machine learning (ML) con comandos de SQL conocidos
Amazon Redshift opera con Amazon SageMaker Autopilot para obtener automáticamente el modelo más adecuado y hacer que la función de predicción esté disponible en Amazon Redshift.



Crea, entrene e implemente modelos de machine learning (ML) con comandos de SQL conocidos
Amazon Redshift opera con Amazon SageMaker Autopilot para obtener automáticamente el modelo más adecuado y hacer que la función de predicción esté disponible en Amazon Redshift.





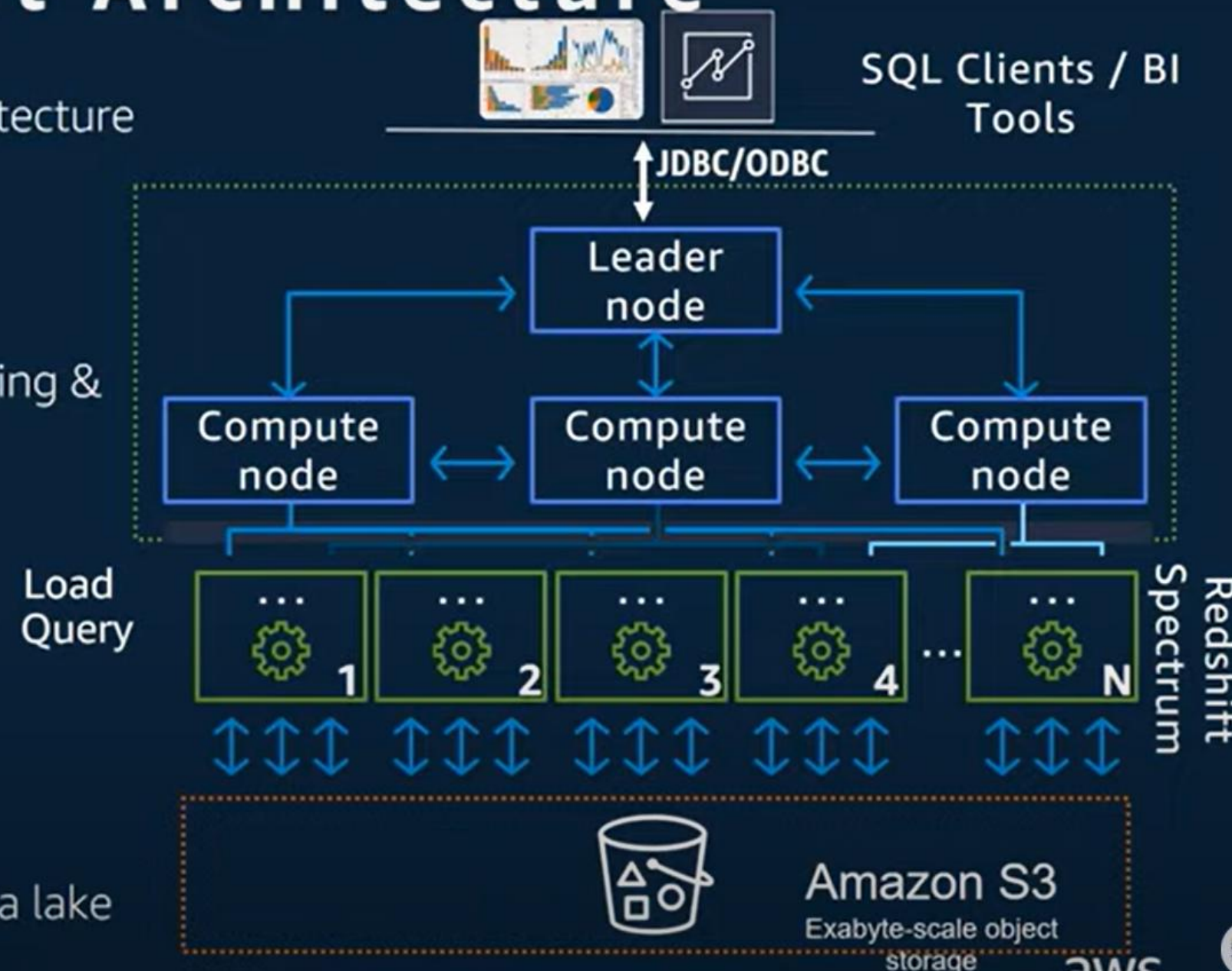
Amazon Redshift Architecture

Getting Started with Amazon Redshift - AWS Online Tech Talks

Amazon Redshift Architecture

Massively parallel, shared nothing architecture

- Leader node
 - SQL endpoint
 - Stores metadata
 - Coordinates parallel SQL processing & ML optimizations
- Compute nodes
 - Local, columnar storage
 - Executes queries in parallel
 - Load, unload, backup, restore from S3
- Amazon Redshift Spectrum nodes
Execute queries directly against data lake



Arquitectura del sistema de almacenamiento de datos - Amazon Redshift

https://youtu.be/_y0YHG95MI48?t=124

Minuto 2

Amazon Redshift slices

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Terminology and Concepts: Slices

A *slice* can be thought of like a virtual compute node

- Unit of data partitioning
- Parallel query processing

Facts about slices:

- Each compute node has either 2, 16, or 32 slices
- Table rows are distributed to slices
- A slice processes only its own data

<https://youtu.be/Hur-p3kGDTA?t=1009> min
16:49

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved.

▶ ⏮ 🔊 17:06 / 51:05 • Best Practices: Sort Keys > Desliza hacia abajo para ver más detalles

aws

Amazon Redshift Blocks

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Terminology and Concepts: Blocks

Presiona **Esc** para salir de pantalla completa

Column data is persisted to 1 MB immutable blocks

Blocks are individually encoded with 1 of 11 encodings

A full block can contain millions of values

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved.

Desliza hacia abajo para ver más detalles

aws

10:47 / 51:05 • Terminology and Concepts: Blocks

<https://youtu.be/Hur-p3kGDTA?t=652> minuto 11:23

[Amazon Redshift: 3 Comprehensive Aspects \(hevodata.com\)](https://hevodata.com)

Amazon Redshift Zone Maps

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Terminology and Concepts: Zone Maps

Goal: eliminates unnecessary I/O

In-memory block metadata

- Contains per-block min and max values
- All blocks automatically have zone maps
- Effectively prunes blocks which cannot contain data for a given query

<https://youtu.be/13ilj34nkQE>

Amazon Redshift is columnar

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Terminology and Concepts: Columnar

Amazon Redshift uses a columnar architecture for storing data on disk

Goal: reduce I/O for analytics queries

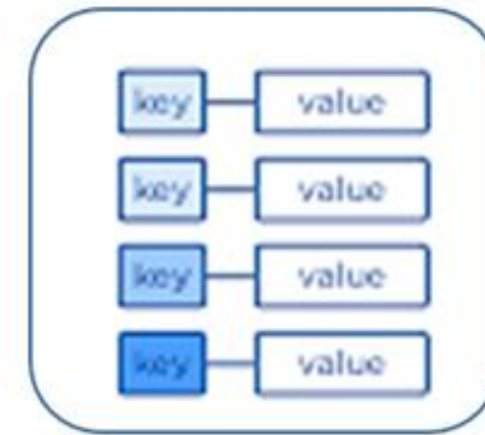
Physically store data on disk by column rather than row

Only read the column data that is required

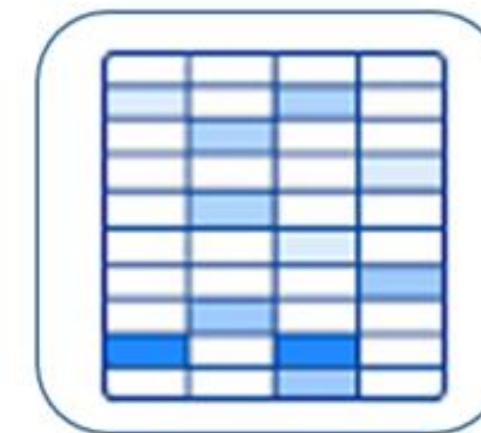
© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved. Desliza hacia abajo para ver más detalles



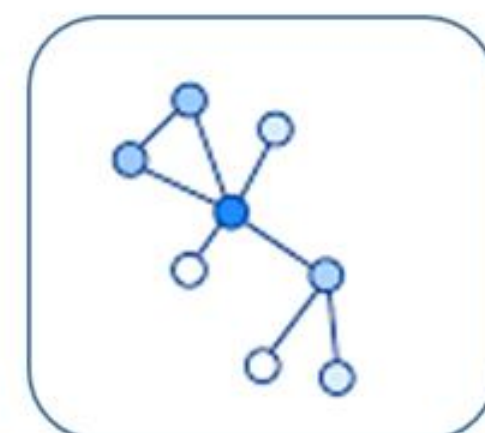
Document
Store



Key-Value
Store



Wide-Column
Store



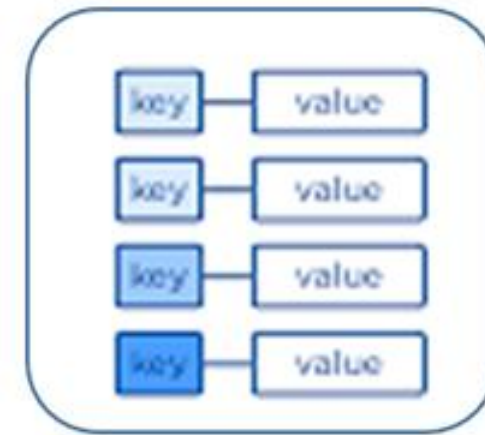
Graph
Store

Amazon Redshift is columnar

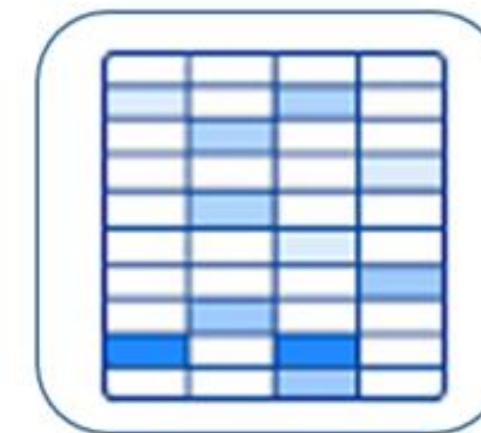
Almacenamiento en columnas - Amazon Redshift



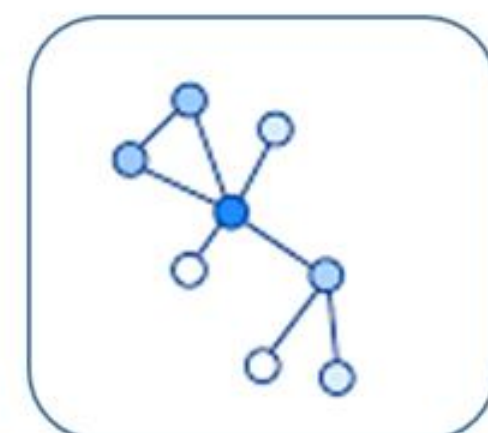
Document Store



Key-Value Store



Wide-Column Store



Graph Store

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Columnar Architecture: Example

```
CREATE TABLE deep_dive (
  aid INT      --audience_id
,loc CHAR(3)  --location
,dt DATE      --date
);
```

aid	loc	dt
1	SFO	2017-10-20
2	JFK	2017-10-20
3	SFO	2017-04-01
4	JFK	2017-05-14

Row-based storage

- Need to read everything
- Unnecessary I/O

SELECT min(dt) FROM deep_dive;

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved. Desliza hacia abajo para ver más detalles

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Columnar Architecture: Example

```
CREATE TABLE deep_dive (
  aid INT      --audience_id
,loc CHAR(3)  --location
,dt DATE      --date
);
```

aid	loc	dt
1	SFO	2017-10-20
2	JFK	2017-10-20
3	SFO	2017-04-01
4	JFK	2017-05-14

Column-based storage

- Only scan blocks for relevant column

SELECT min(dt) FROM deep_dive;

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved.

Amazon Redshift compression

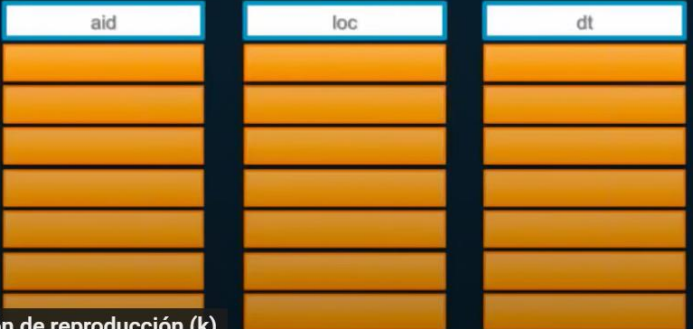
Deep Dive on Amazon Redshift - AWS Online Tech Talks

Compression: Example

```
CREATE TABLE deep_dive (  
  aid INT      --audience_id  
  ,loc CHAR(3) --location  
  ,dt  DATE    --date  
);
```

aid	loc	dt
1	SFO	2017-10-20
2	JFK	2017-10-20
3	SFO	2017-04-01
4	JFK	2017-05-14

Add 1 of 11 different encodings to each column



Botón de reproducción (k)

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved. Desliza hacia abajo para ver más detalles

aws

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Compression: Example

```
CREATE TABLE deep_dive (  
  aid INT      ENCODE ZSTD  
  ,loc CHAR(3) ENCODE BYTEDICT  
  ,dt  DATE    ENCODE RUNLENGTH  
);
```

aid	loc	dt
1	SFO	2017-10-20
2	JFK	2017-10-20
3	SFO	2017-04-01
4	JFK	2017-05-14

More efficient compression is due to storing the same data type in the columnar architecture

Columns grow and shrink independently

Reduces storage requirements

Reduces I/O



© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved.

aws

Amazon Redshift data sorting

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Terminology and Concepts: Data Sorting

Presiona **Esc** para salir de pantalla completa

Goal: make queries run faster by increasing the effectiveness of zone maps and reducing I/O

Impact: enables range-restricted scans to prune blocks by leveraging zone maps

Achieved with a SORTKEY defined on one or more columns

Optimal sort key is dependent on:

- Query patterns
- Business requirements
- Data profile

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved. Desliza hacia abajo para ver más detalles

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Sort Key: Example

Presiona **Esc** para salir de pantalla completa

Add a sort key to one or more columns to physically sort the data on disk

```
CREATE TABLE deep_dive (
  aid INT      --audience_id
  ,loc CHAR(3) --location
  ,dt  DATE    --date
) SORT KEY (dt, loc);
```

deep_dive		
aid	loc	dt
1	SFO	2017-10-20
2	JFK	2017-10-20
3	SFO	2017-04-01
4	JFK	2017-05-14

deep_dive (sorted)		
aid	loc	dt
3	SFO	2017-04-01
4	JFK	2017-05-14
2	JFK	2017-10-20
1	SFO	2017-10-20

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved. Desliza hacia abajo para ver más detalles

<https://youtu.be/Hur-p3kGDTA?t=720>

Amazon Redshift data distribution

Deep Dive on Amazon Redshift - AWS Online Tech Talks

Data Distribution

Distribution style is a table property which dictates how that table's data is distributed throughout the cluster:

- **KEY:** value is hashed, same value goes to same location (slice)
- **ALL:** full table data goes to the first slice of every node
- **EVEN:** round robin

Goals:

- Distribute data evenly for parallel processing
- Minimize data movement during query processing

key and what we essentially do is we take

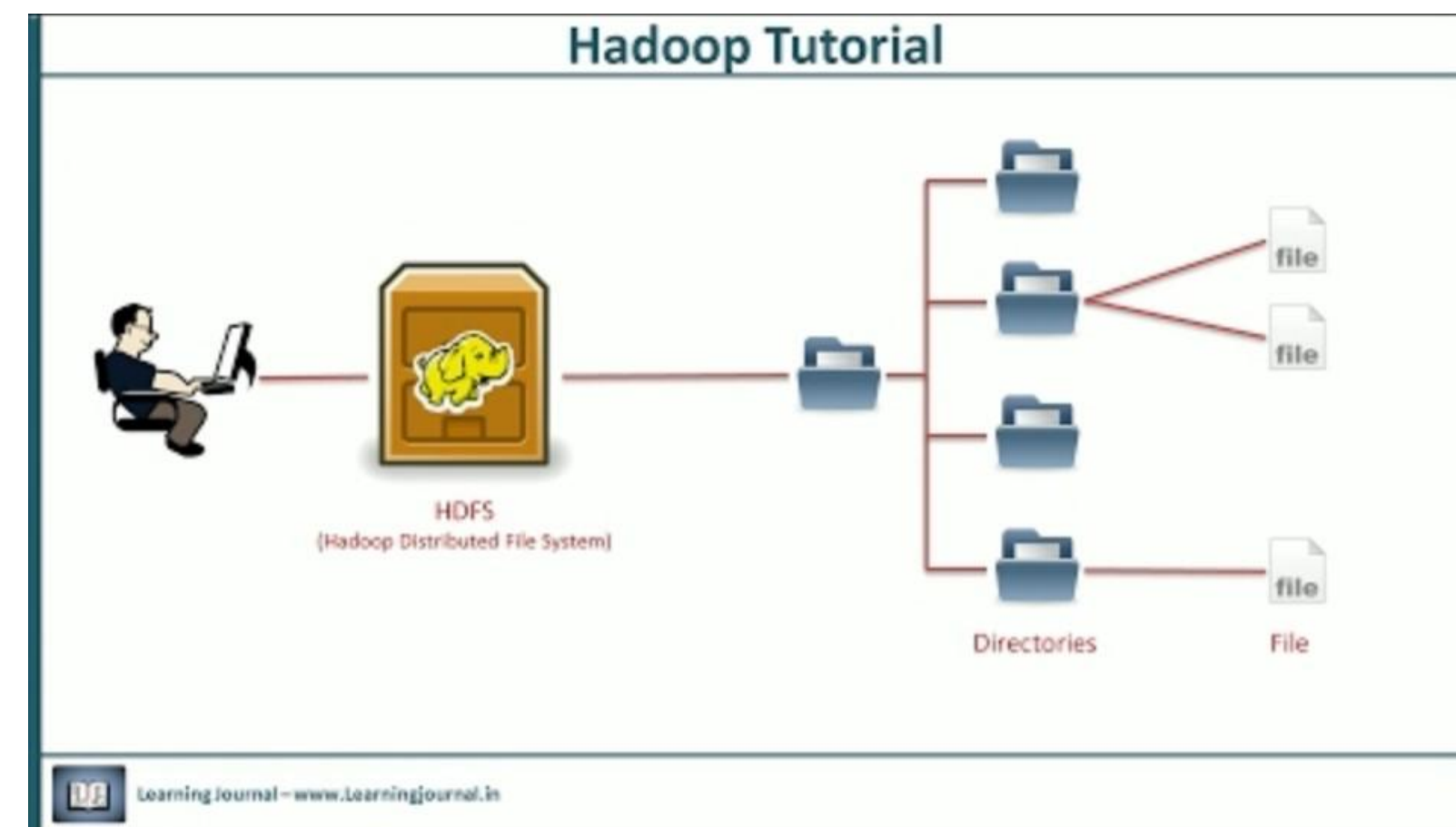
© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved.
17:51 / 51:05 • Best Practices: Sort Keys

<https://youtu.be/Hur-p3kGDTA?t=1071>

1.1 ¿Qué es HDFS?

HDFS (Hadoop Distributed File System) es el sistema de archivos distribuido del proyecto **Apache Hadoop**, diseñado para almacenar y gestionar grandes volúmenes de datos (Big Data) de forma eficiente y segura en un entorno distribuido. A diferencia de los sistemas de archivos tradicionales, HDFS permite el almacenamiento de datos en un clúster de máquinas, con capacidades de escalabilidad y alta disponibilidad.

- **Replicación de datos.** HDFS hace que los datos estén siempre disponibles, aunque falle uno de los ordenadores del clúster. Gracias a este sistema se pueden replicar datos y seguir su procesamiento.
- **Tolerancia a fallos.** La capacidad de replicación y de copia del sistema de archivos distribuidos hace que el sistema sea más tolerante a posibles fallos.
- **Escalabilidad.** Al funcionar a través de un clúster de ordenadores, es mucho más sencillo escalar la capacidad de procesamiento añadiendo o quitando ordenadores.
- **Rendimiento.** Al almacenar los datos en un sistema distribuido, HDFS hace que se puedan procesar de forma paralela. Esto hace que se reduzca el tiempo de procesamiento y, por tanto, se mejore el rendimiento en el mismo.

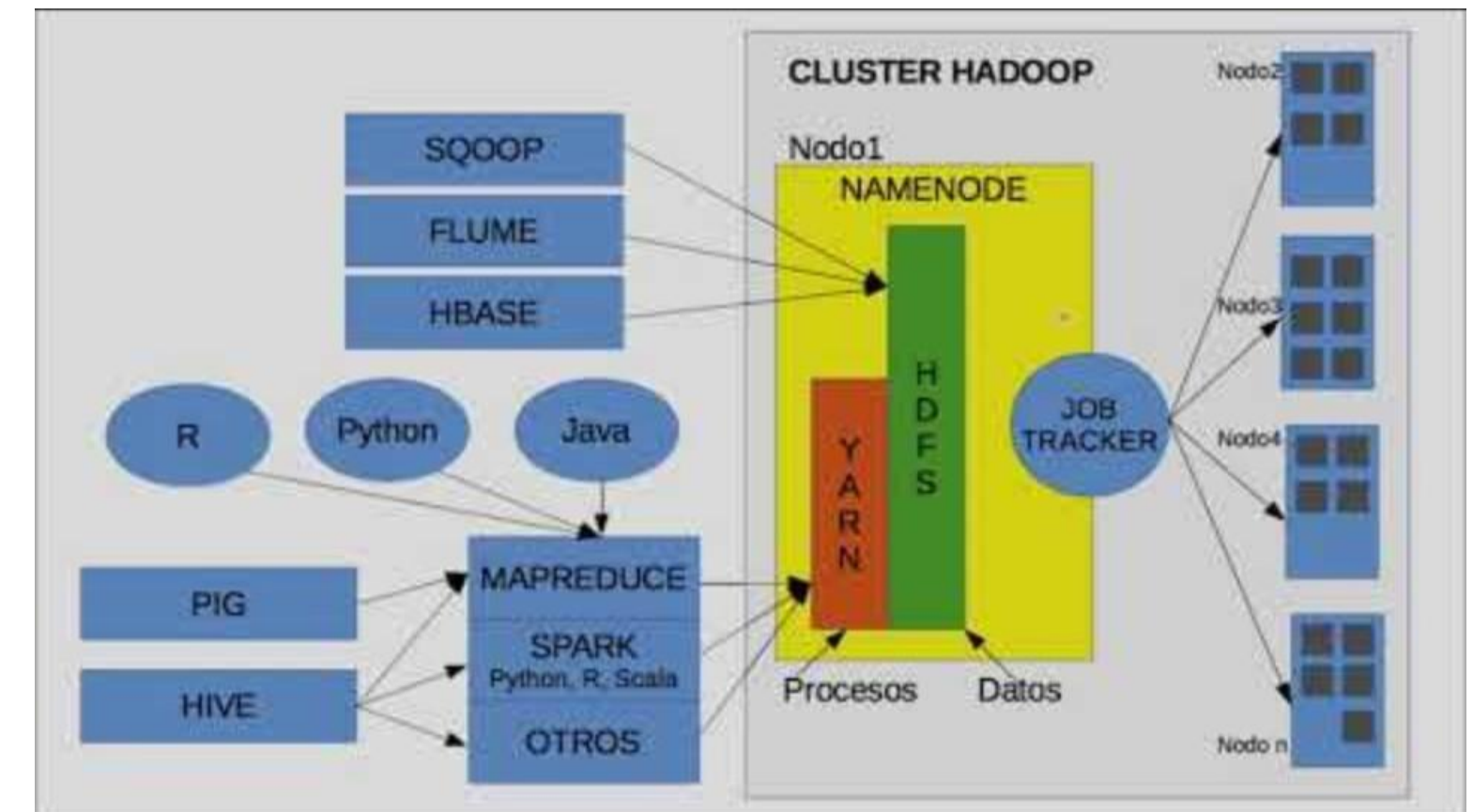


HDFS sigue un modelo maestro-esclavo, donde:

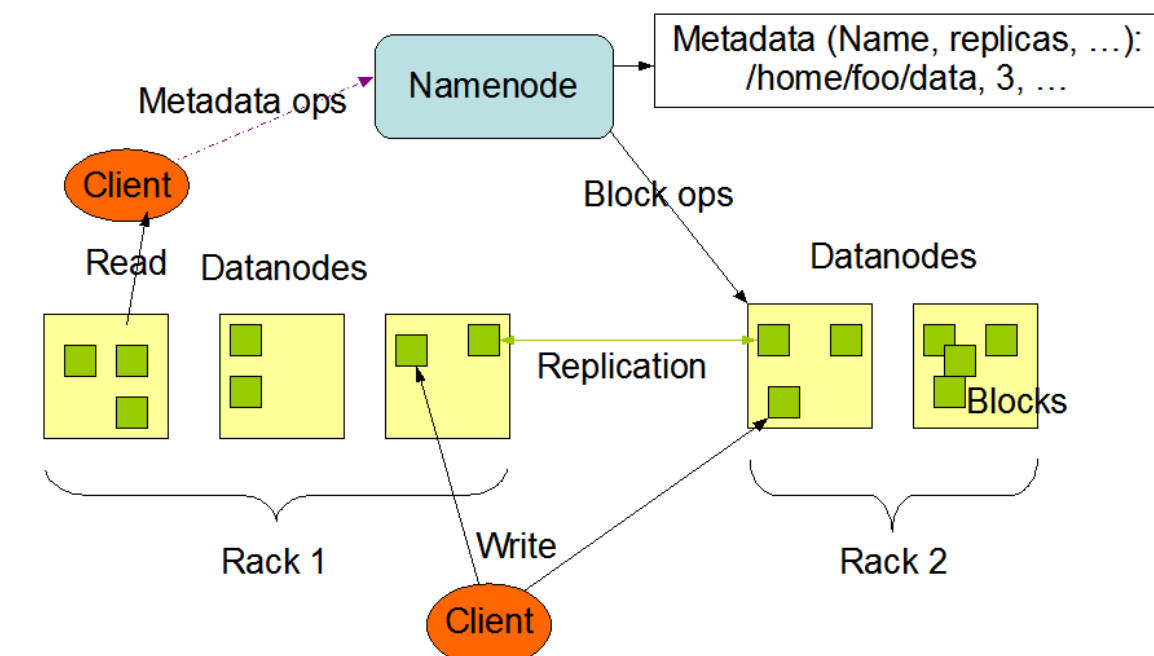
- El **NameNode** es el servidor central que administra el sistema de archivos y conoce la ubicación de los bloques de datos.
- Los **DataNodes** almacenan los datos reales en bloques y ejecutan las operaciones de lectura/escritura solicitadas por los clientes.

Características principales:

- **Distribución de datos:** Los archivos se dividen en bloques (típicamente de 128 MB) que se distribuyen a través de los DataNodes.
- **Replicación:** Cada bloque tiene múltiples copias distribuidas en varios nodos para garantizar la disponibilidad en caso de fallo.



HDFS Architecture



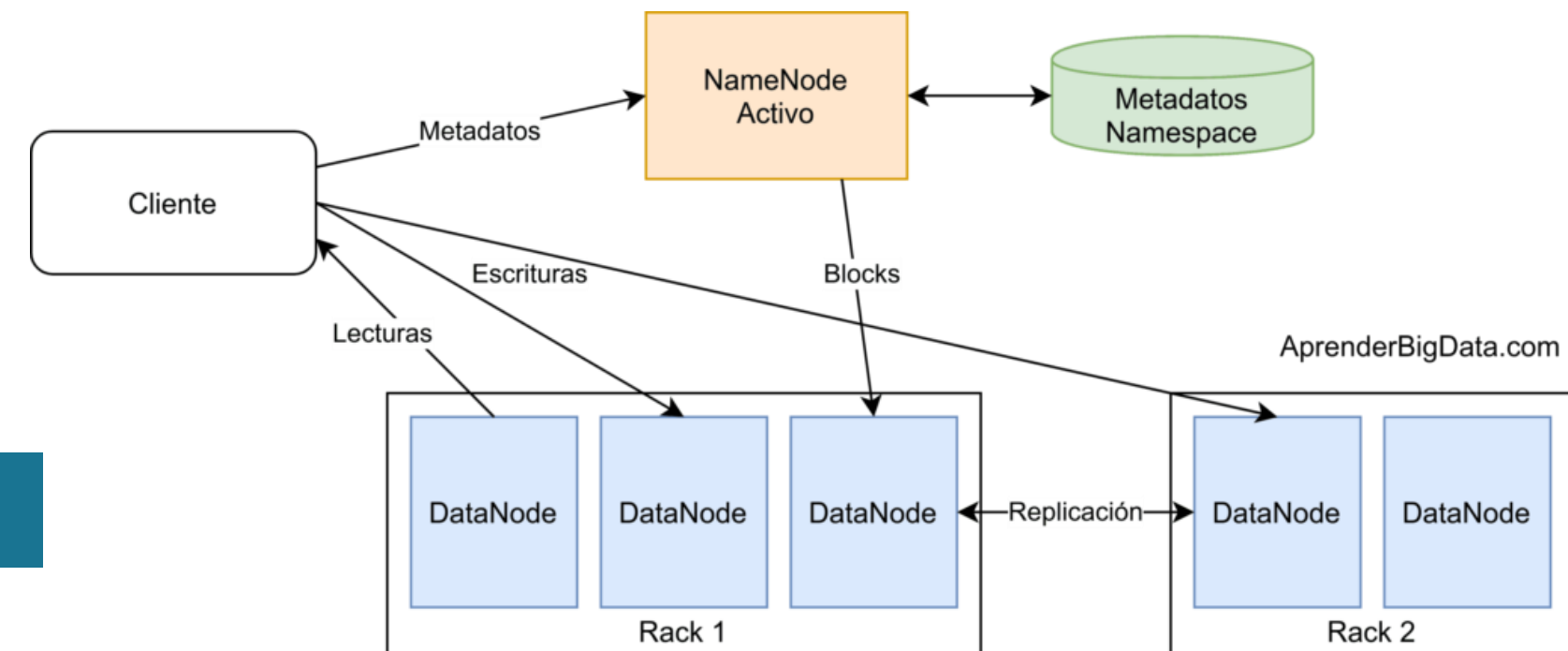
NameNode

- El NameNode es el núcleo de la arquitectura HDFS. Administra el sistema de archivos y el namespace, rastreando dónde se almacenan los bloques en el clúster. No almacena los datos reales, sino la **metadata** (información de ubicación de los bloques y estructura de archivos).
- **Funciones del NameNode:**
 - Gestionar la creación, eliminación y modificación de archivos y directorios.
 - Mantener un mapa de dónde se encuentran los bloques de datos en los DataNodes.
- **Alta disponibilidad:** Si el NameNode falla, puede ser una vulnerabilidad crítica. Por ello, se utilizan NameNodes secundarios (o standby) para tener redundancia.

DataNode

Los DataNodes son los encargados de almacenar los datos reales. Los archivos se dividen en bloques y se distribuyen entre varios DataNodes.

- **Funciones del DataNode:**
 - Almacenar y replicar bloques de datos.
 - Realizar operaciones de lectura y escritura según lo soliciten los clientes.
 - Informar periódicamente al NameNode sobre el estado de los bloques que contienen.
- **2.3 Bloques y replicación**
- HDFS divide los archivos en **bloques** de tamaño fijo (128 MB por defecto), lo que permite una distribución eficiente de los datos entre diferentes nodos del clúster.
- **Replicación:** Cada bloque se replica en varios DataNodes (típicamente tres copias por bloque) para asegurar la tolerancia a fallos. Si un nodo falla, los datos aún pueden ser accesibles desde otras réplicas.



Airbnb

Airbnb almacena y procesa grandes volúmenes de datos sobre reservas, interacciones de usuarios, precios, y ubicaciones utilizando Hadoop. Los casos de uso incluyen:

- **Análisis de datos de usuarios y reservas** para optimizar la estrategia de precios dinámicos y maximizar el ingreso para los anfitriones.
- **Modelos de machine learning** para mejorar las recomendaciones de propiedades a los usuarios, basadas en el análisis del comportamiento anterior.
- **Mejoras en la búsqueda de propiedades:** Utiliza Hadoop para analizar las consultas de búsqueda y mejorar los algoritmos que sugieren los lugares más relevantes.



Netflix

Netflix usa Hadoop para gestionar y analizar datos masivos provenientes de las interacciones de los usuarios con su plataforma. Los principales casos de uso incluyen:

- **Recomendaciones personalizadas:** Netflix utiliza Hadoop para analizar el historial de visualizaciones y patrones de comportamiento de los usuarios, mejorando sus algoritmos de recomendación de contenido.
- **Análisis predictivo:** Hadoop permite a Netflix predecir qué contenido será popular y ajustar su catálogo de películas y series en consecuencia.
- **Optimización de la infraestructura de red:** Analizando datos de streaming en tiempo real para mejorar la calidad de video y reducir tiempos de buffering.



Gracias